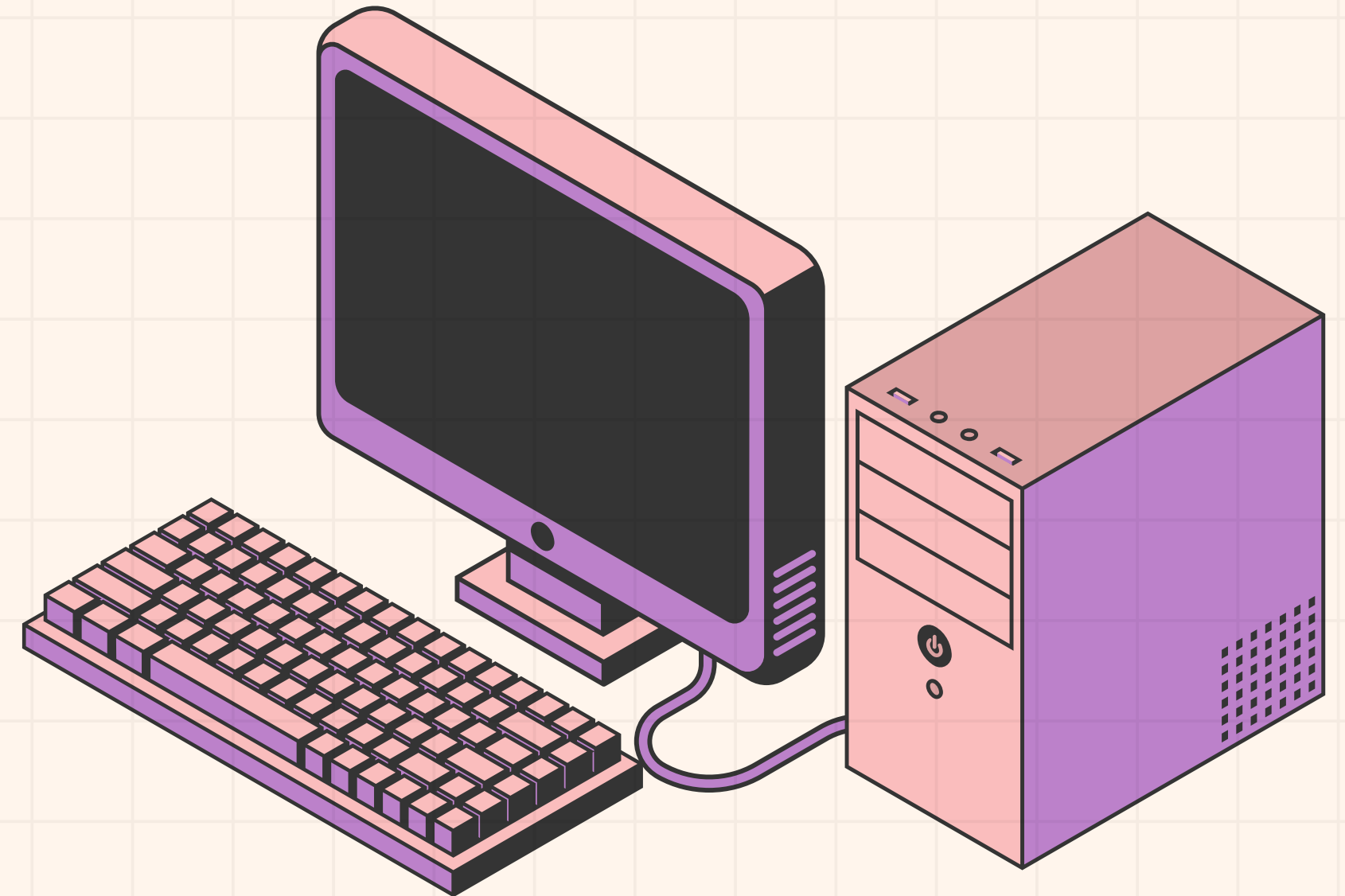


SEMINAR PERFORMANCE TESTING



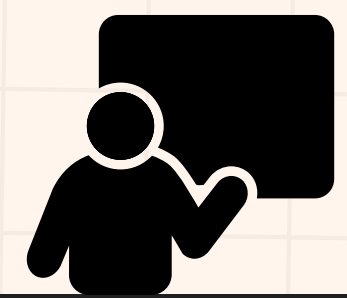
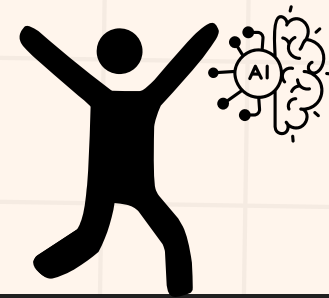
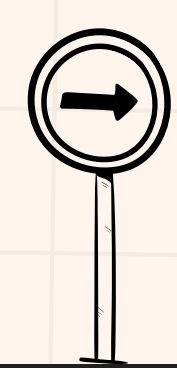
PERFORMANCE TESTING

GIỚI THIỆU

LÝ THUYẾT

JMETER

DEMO



1

TỔNG QUAN VỀ
PERFORMANCE
TESTING



2

AI TRONG
PERFORMANCE
TESTING



3

APACHE JMETER



4

DEMO

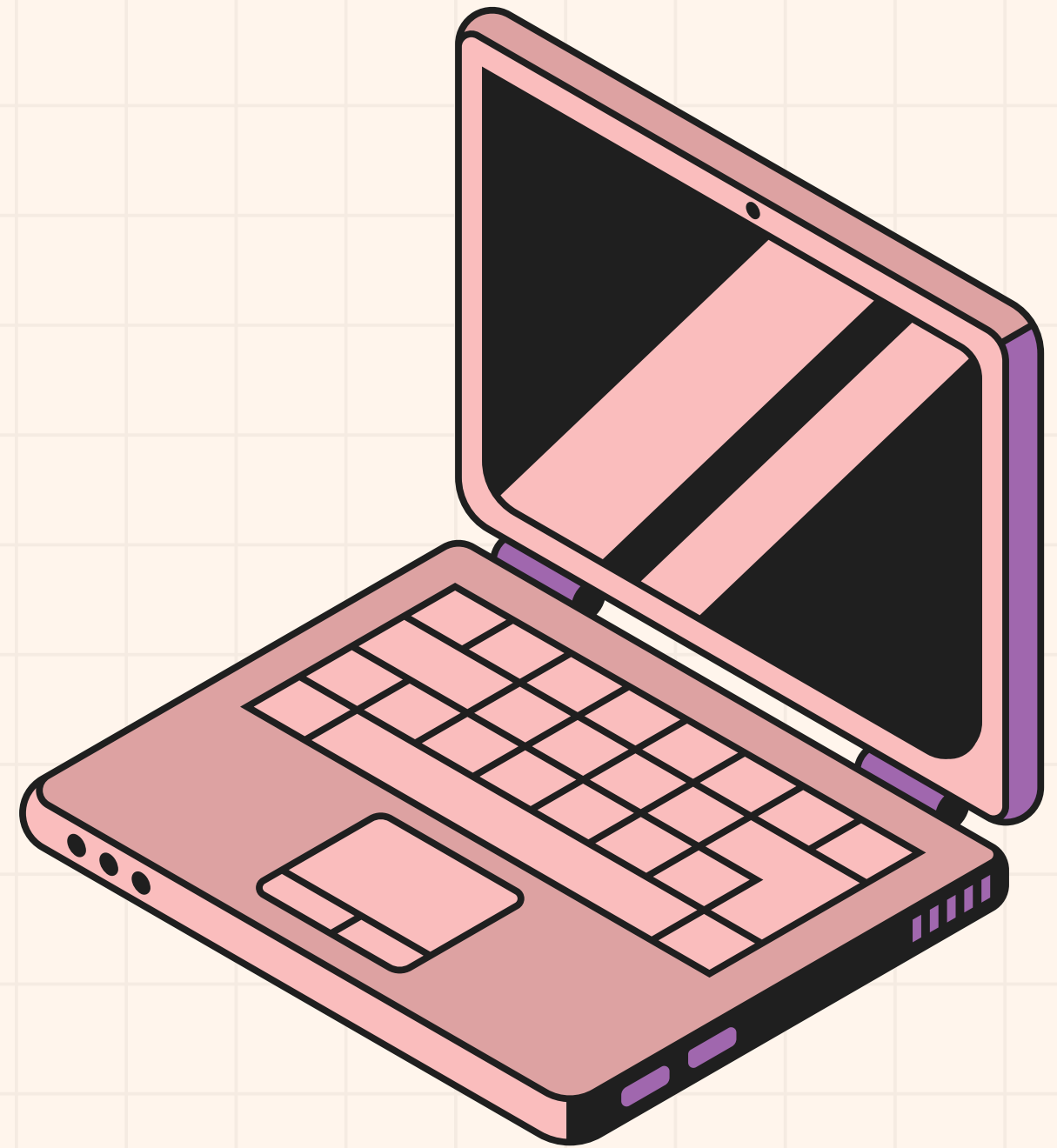
[GIỚI THIỆU](#)

[LÝ THUYẾT](#)

[JMETER](#)

[DEMO](#)

TỔNG QUAN VỀ KIỂM THỬ HIỆU NĂNG



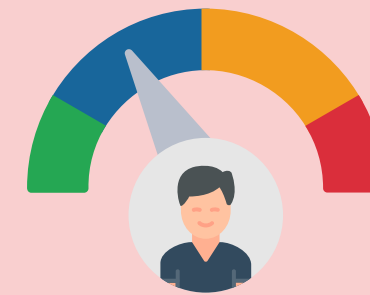
I. TỔNG QUAN VỀ KIỂM THỬ HIỆU NĂNG

1. Kiểm Thử Hiệu Năng là gì?

1.1. Kiểm Thử Phi Chức Năng

Kiểm thử phi chức năng (Non-Functional Testing) là quá trình đánh giá các khía cạnh không liên quan đến chức năng của phần mềm.

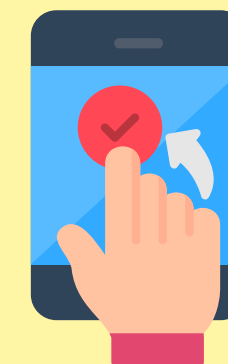
Mục tiêu của kiểm thử phi chức năng là đảm bảo rằng phần mềm hoạt động tốt không chỉ từ góc nhìn chức năng mà còn từ các khía cạnh khác.



**Hiệu suất
(Performance Testing)**



**Bảo mật
(Security Testing)**



**Tương tác người dùng
(Usability Testing)**



**Tương thích
(Compatibility Testing)**

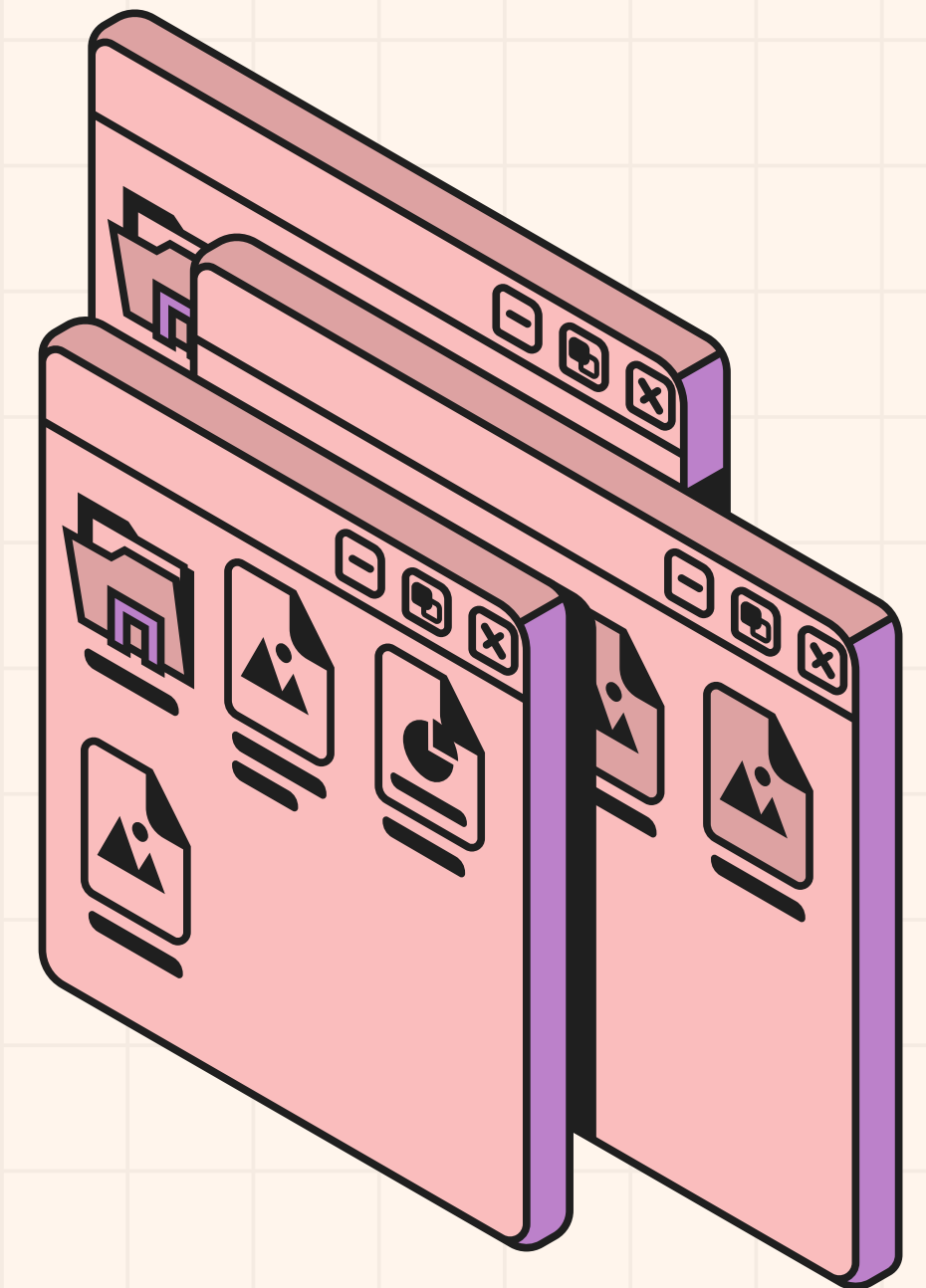
I. TỔNG QUAN VỀ KIỂM THỬ HIỆU NĂNG

1. Kiểm Thử Hiệu Năng là gì?

1.2. Kiểm Thử Hiệu Năng

Kiểm thử hiệu năng (Performance Testing) là quá trình kiểm tra xem một chương trình phần mềm có chạy nhanh không, phản hồi có nhanh không, có chịu tải tốt không, sử dụng tài nguyên như thế nào và có thể mở rộng đáp ứng nhiều người dùng cùng lúc hay không.

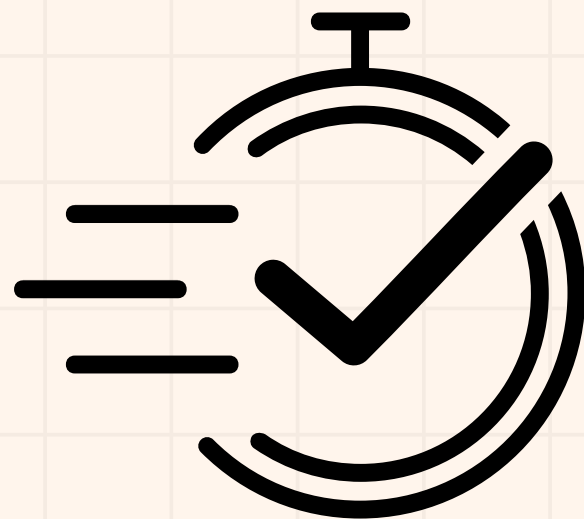
Nói đơn giản, **mục tiêu của kiểm thử hiệu năng** là để phát hiện và loại bỏ những điểm làm phần mềm chạy chậm, bị nghẽn, hoặc không đáp ứng được hiệu suất như mong muốn.



I. TỔNG QUAN VỀ KIỂM THỬ HIỆU NĂNG

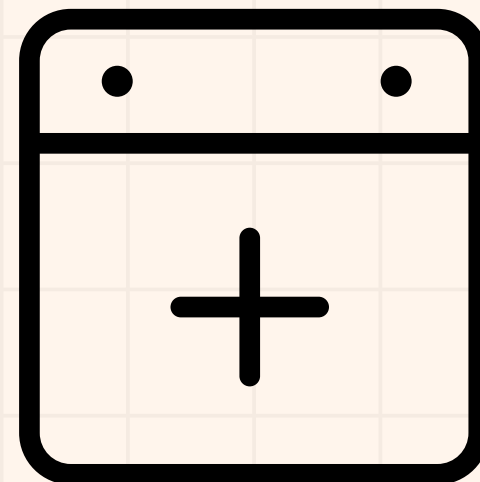
1. Kiểm Thử Hiệu Năng là gì?

1.3. Mục tiêu của Kiểm Thử Hiệu Năng



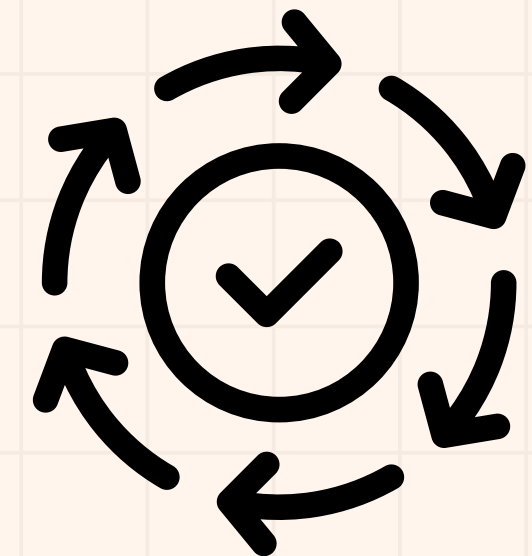
TỐC ĐỘ

Xác định xem ứng dụng có phản hồi nhanh không



KHẢ NĂNG MỞ RỘNG

Xác định tải người dùng tối đa mà ứng dụng phần mềm có thể xử lý



TÍNH ỔN ĐỊNH

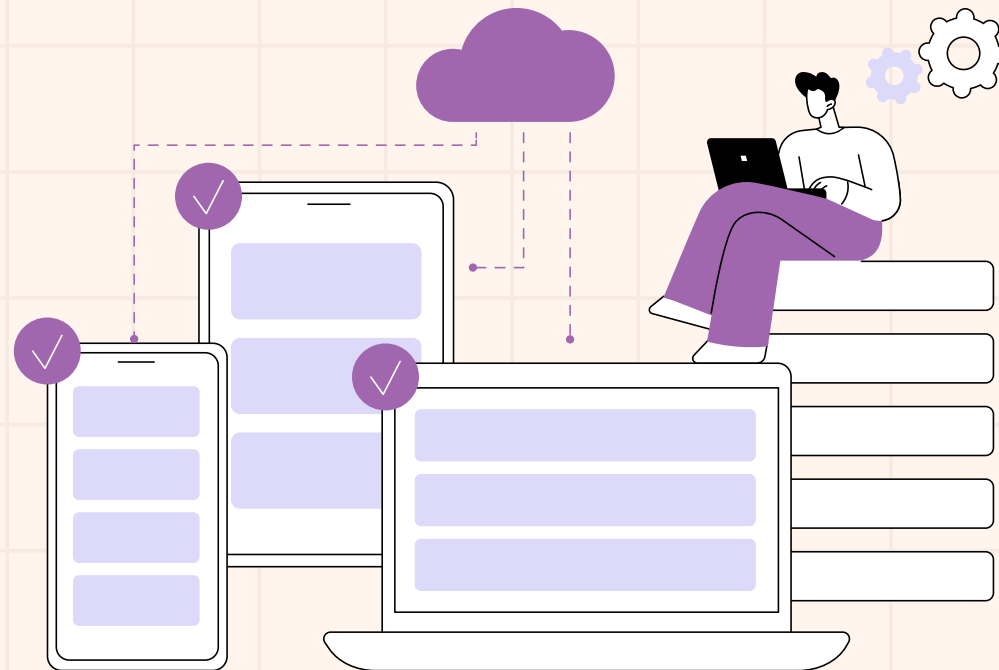
Xác định xem ứng dụng có ổn định dưới các mức tải khác nhau không

I. TỔNG QUAN VỀ KIỂM THỬ HIỆU NĂNG

1. Kiểm Thử Hiệu Năng là gì?

1.3. Mục tiêu của Kiểm Thử Hiệu Năng

Performance Testing giúp xác định tắc nghẽn hệ thống, thiết lập đường cơ sở cho các lần kiểm thử sau, hỗ trợ tối ưu hiệu suất, đánh giá mức độ đáp ứng yêu cầu và cung cấp dữ liệu để các bên ra quyết định về chất lượng ứng dụng.



Ngoài ra, các kết quả từ việc kiểm thử hiệu suất và phân tích có thể giúp ước tính cấu hình phần cứng cần thiết để hỗ trợ các ứng dụng khi đưa sản phẩm đi vào sử dụng rộng rãi.

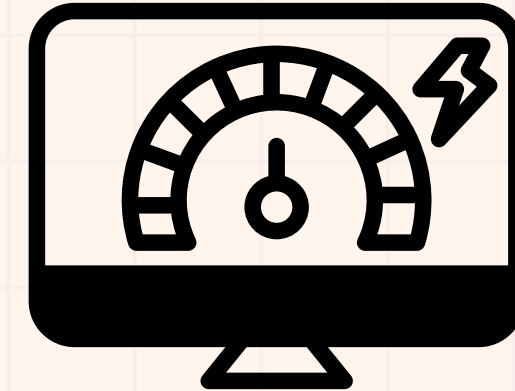
I. TỔNG QUAN VỀ KIỂM THỬ HIỆU NĂNG

1. Kiểm Thử Hiệu Năng là gì?

1.4. Các loại Kiểm Thử Hiệu Năng

LOAD TESTING

Load testing được thực hiện để đánh giá hành vi của hệ thống khi tăng tải (số người dùng đồng thời, số lượng transaction/request/...), để xác định mức tải tối đa (maximum load/ngưỡng) mà hệ thống có thể xử lý.



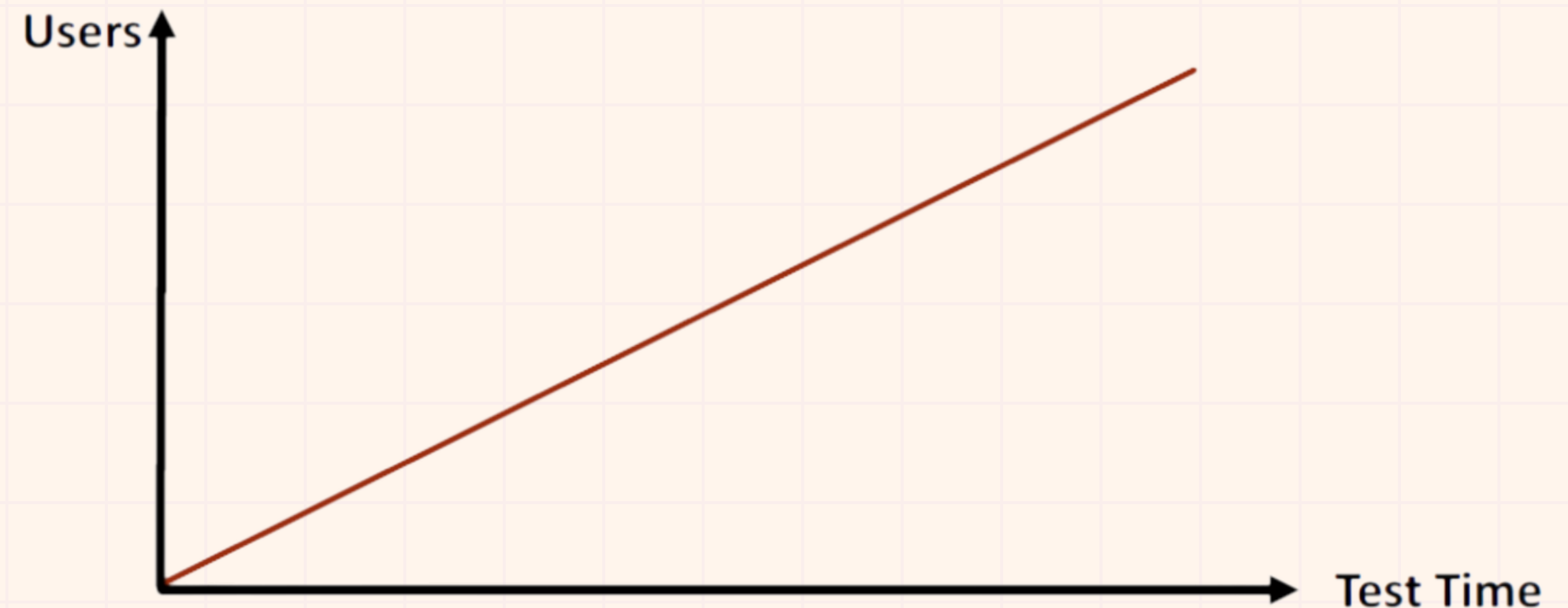
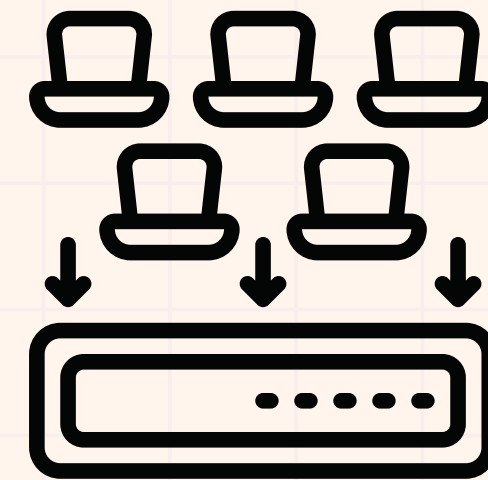
I. TỔNG QUAN VỀ KIỂM THỬ HIỆU NĂNG

1. Kiểm Thử Hiệu Năng là gì?

1.4. Các loại Kiểm Thử Hiệu Năng

STRESS TESTING

Stress testing được thực hiện để đánh giá hành vi của hệ thống khi tải vượt ngưỡng, tìm ra điểm chết và đánh giá khả năng phục hồi lại của hệ thống.



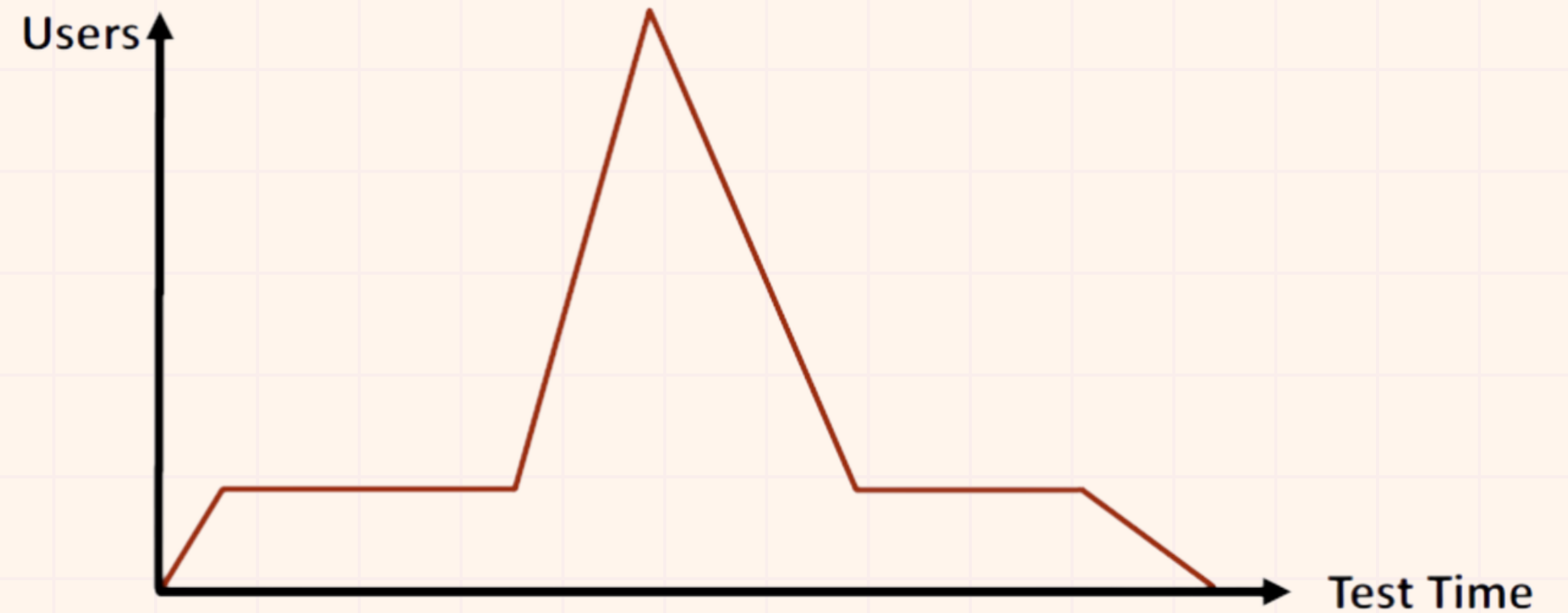
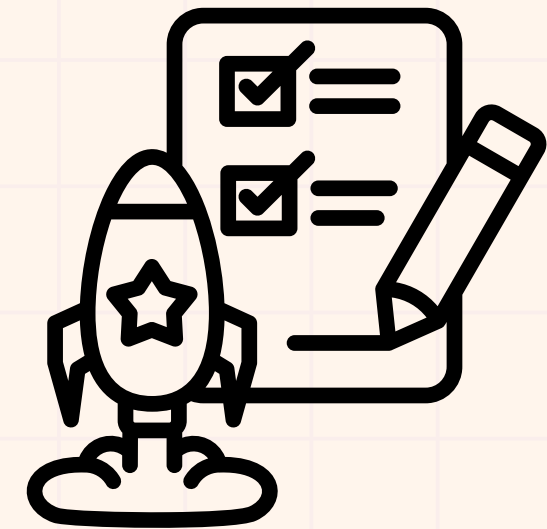
I. TỔNG QUAN VỀ KIỂM THỬ HIỆU NĂNG

1. Kiểm Thử Hiệu Năng là gì?

1.4. Các loại Kiểm Thử Hiệu Năng

SPIKE TESTING

Spike testing được thực hiện để đánh giá hành vi của hệ thống khi tải tăng cao đột ngột trong thời gian ngắn.



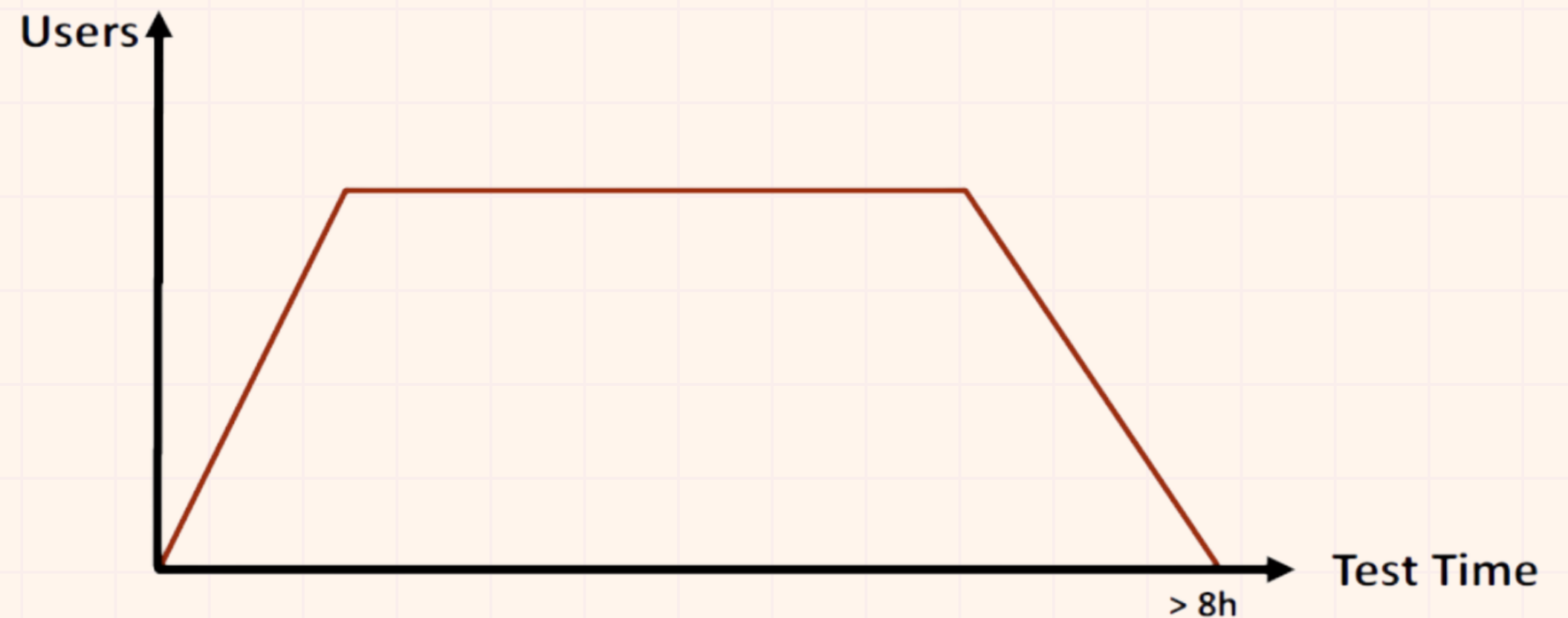
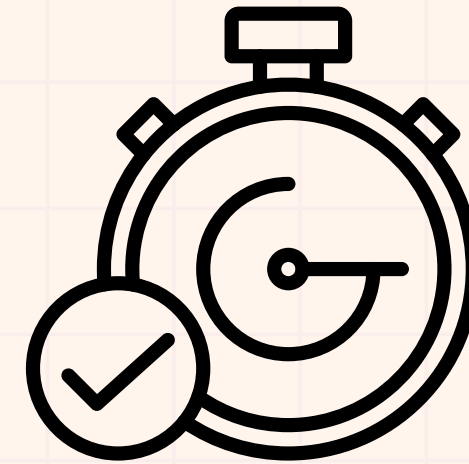
I. TỔNG QUAN VỀ KIỂM THỬ HIỆU NĂNG

1. Kiểm Thử Hiệu Năng là gì?

1.4. Các loại Kiểm Thử Hiệu Năng

ENDURANCE TESTING

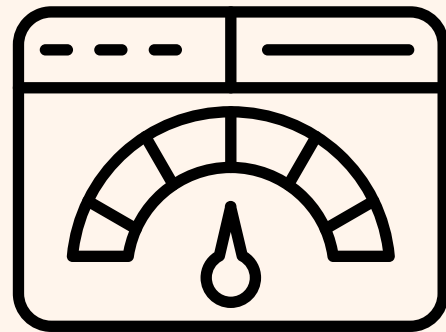
Thực hiện trong thời gian dài để đánh giá việc sử dụng tài nguyên (memory) của hệ thống. Thông thường, sẽ chạy với 70-80% ngưỡng của hệ thống trong thời gian 3-11h.



I. TỔNG QUAN VỀ KIỂM THỬ HIỆU NĂNG

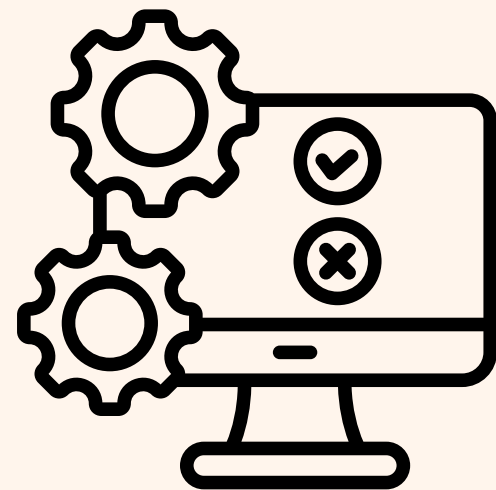
1. Kiểm Thử Hiệu Năng là gì?

1.4. Các loại Kiểm Thử Hiệu Năng



SCALABILITY TESTING

Thu thập các số liệu cần thiết để dự báo và xác định cấu hình phần cứng phù hợp, đảm bảo hệ thống có thể đáp ứng được nhu cầu sử dụng lớn hơn trong tương lai.

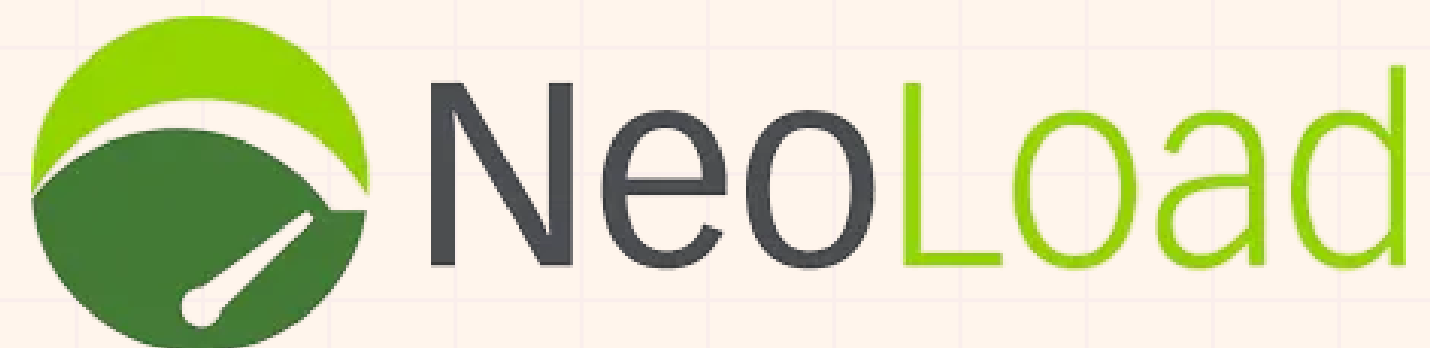


VOLUME TESTING

Kiểm tra xem hệ thống có còn hoạt động ổn định, nhanh chóng và không xảy ra lỗi khi phải truy xuất, xử lý, báo cáo hoặc đồng bộ với **dữ liệu khổng lồ**.

I. TỔNG QUAN VỀ KIỂM THỬ HIỆU NĂNG

2. Một số công cụ kiểm thử hiệu năng



I. TỔNG QUAN VỀ KIỂM THỬ HIỆU NĂNG

2. Một số công cụ kiểm thử hiệu năng



- Là công cụ kiểm thử hiệu suất **mã nguồn mở** do Apache phát triển, được viết bằng Java.
- Kiểm tra **độ bền, hiệu suất và khả năng chịu tải** của hệ thống.
- Hỗ trợ kiểm thử cho các ứng dụng **Web, API, cơ sở dữ liệu, máy chủ,...**
- **Nhiều giao thức:** HTTP, FTP, JDBC, SOAP,...
- Có thể mở rộng chức năng thông qua hệ thống plugin **phong phú**.

I. TỔNG QUAN VỀ KIỂM THỬ HIỆU NĂNG

2. Một số công cụ kiểm thử hiệu năng



- Là công cụ kiểm thử hiệu suất **mã nguồn mở**.
- Được xây dựng bằng **Go** và viết kịch bản bởi **JavaScript**.
- Hỗ trợ giao diện dòng lệnh, dễ tích hợp vào CI/CD pipeline.
- Tập trung vào kiểm thử tải cho **API, microservices và ứng dụng web**.
- Ưu điểm **nhẹ, nhanh**, tiêu tốn ít tài nguyên hệ thống.

I. TỔNG QUAN VỀ KIỂM THỬ HIỆU NĂNG

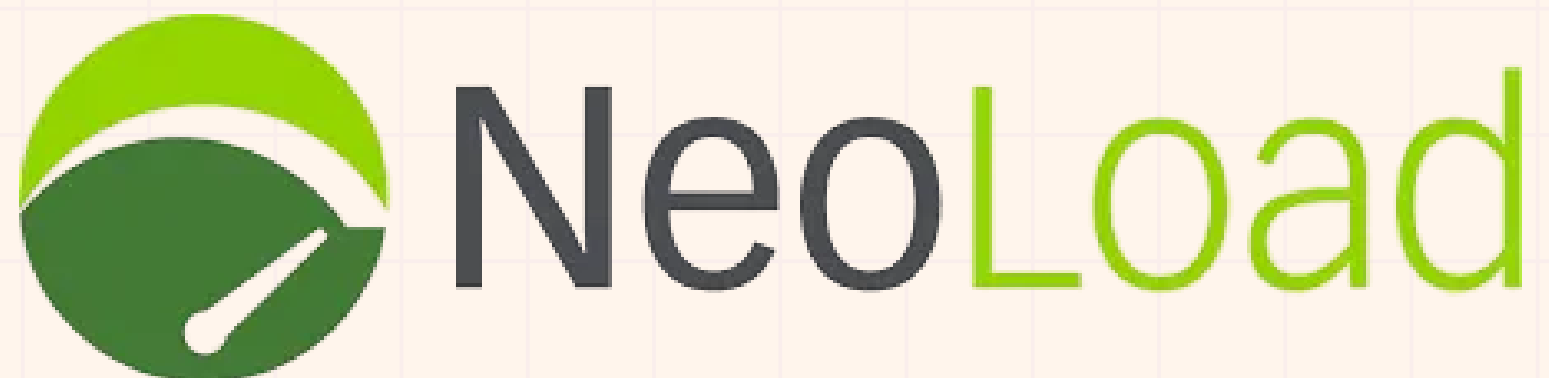
2. Một số công cụ kiểm thử hiệu năng



- Là công cụ kiểm thử hiệu suất **chuyên nghiệp**.
- Giúp **mô phỏng hàng nghìn người dùng ảo** truy cập đồng thời.
- Được sử dụng để đánh giá **hiệu suất và khả năng chịu tải** của hệ thống.
- **Nhiều giao thức**: HTTP, Web Services, SAP, Oracle, ...
- Phù hợp cho các hệ thống **phức tạp và quy mô lớn**.

I. TỔNG QUAN VỀ KIỂM THỬ HIỆU NĂNG

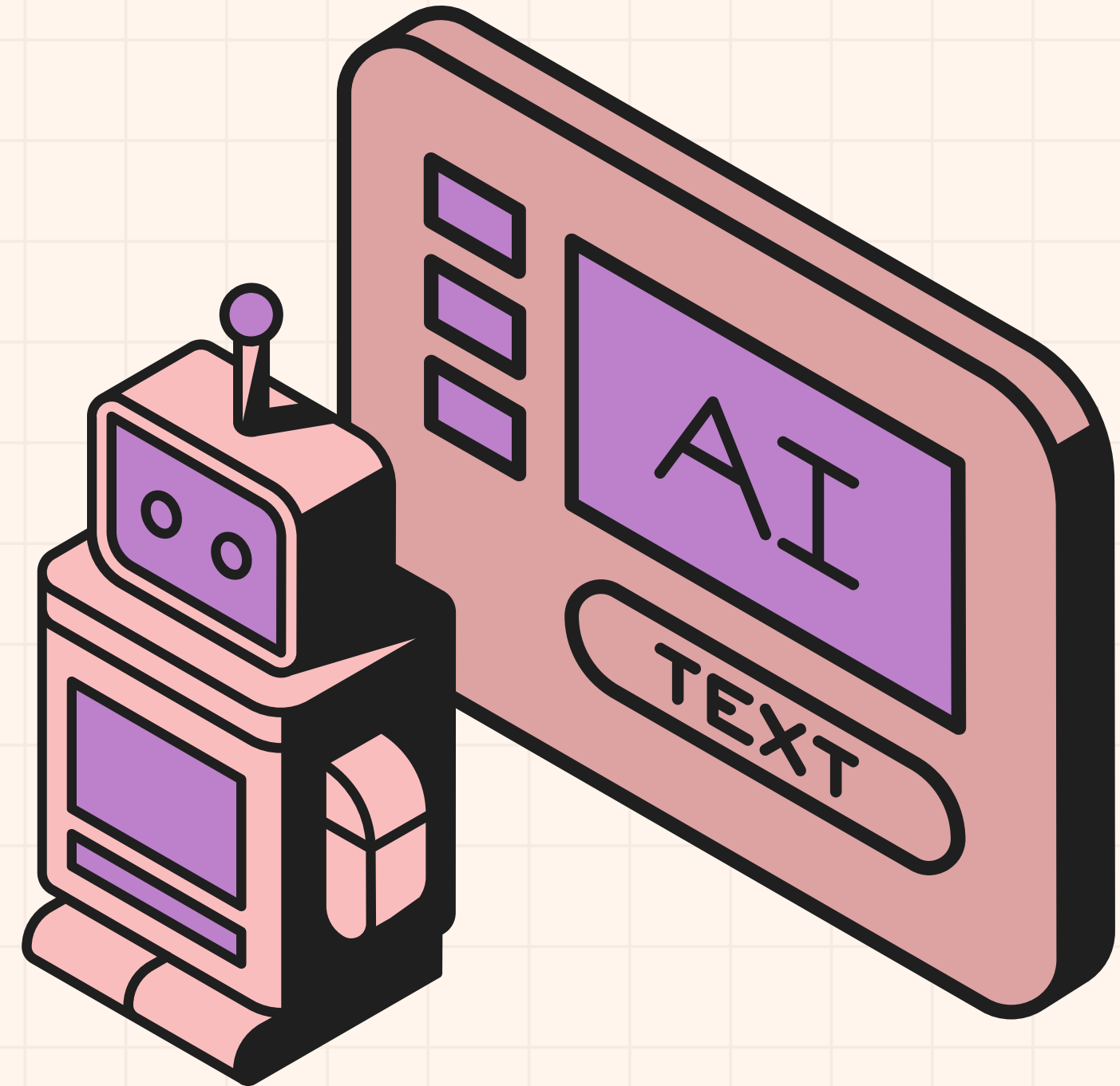
2. Một số công cụ kiểm thử hiệu năng



- Là công cụ kiểm thử hiệu suất **thương mại** do Neotys phát triển.
- Tập trung vào mô phỏng tải lớn cho ứng dụng **web, API và ứng dụng di động**.
- Giao diện GUI **trực quan**, dễ ghi lại và thiết kế kịch bản thử nghiệm.
- Tích hợp **mạnh mẽ** với quy trình CI/CD, phù hợp môi trường DevOps.
- Là giải pháp phù hợp cho doanh nghiệp với quy mô thử nghiệm **vừa và lớn**.

Tiêu chí	JMeter	k6	LoadRunner	NeoLoad
Loại công cụ	Mã nguồn mở, GUI-based	Mã nguồn mở, CLI-based	Thương mại, GUI và script-based	Thương mại, GUI-based
Hiệu suất	Tốn tài nguyên, nhưng tối ưu hóa với plugin và phân tán tải lớn	Hiệu suất cao, tối ưu cho tải lớn	Hiệu suất cao, nhưng yêu cầu phần cứng mạnh mẽ	Hiệu suất tốt, tối ưu cho tải phân tán
Hỗ trợ giao thức	Đa dạng (HTTP, FTP, JDBC, SOAP, v.v.), vượt trội	Chủ yếu HTTP, WebSocket, gRPC	Rộng (HTTP, Web Services, SAP, Oracle, Citrix), chuyên sâu doanh nghiệp	Hỗ trợ rộng (HTTP, SOAP, REST, v.v.), tập trung chủ yếu vào web và API
Dễ sử dụng	Dễ với GUI, hỗ trợ tốt với người mới bắt đầu	Dễ với người đã biết JavaScript, khó với người mới bắt đầu	Khó với người mới, cần học cách sử dụng chuyên sâu	Có GUI, nhưng người dùng cần học cách sử dụng
Tích hợp DevOps	Tích hợp tốt nhưng cần cấu hình thủ công	Tích hợp dễ dàng với CI/CD	Tích hợp mạnh với doanh nghiệp, nhưng phức tạp	Tích hợp tốt với CI/CD, thân thiện
Báo cáo	Tích hợp sẵn, chi tiết qua GUI và plugin	Tùy chỉnh qua CLI hoặc k6 Cloud	Báo cáo chuyên sâu nhưng có trả phí	Báo cáo chi tiết, tích hợp với các công cụ phân tích
Cộng đồng & plugin	Lớn, nhiều plugin đa dạng, hỗ trợ mạnh mẽ	Nhỏ hơn, ít plugin hơn	Cộng đồng hạn chế, phụ thuộc hỗ trợ từ Micro Focus	Cộng đồng vừa phải, hỗ trợ từ Neotys
Chi phí	Hoàn toàn miễn phí, không giới hạn	Miễn phí (local), trả phí cho k6 Cloud	Trả phí cao, yêu cầu cấp phép doanh nghiệp	Trả phí

AI TRONG KIỂM THỬ HIỆU NĂNG



II. AI TRONG KIỂM THỬ HIỆU NĂNG

1. Thách Thức Trong Kiểm Thử Hiệu Suất

Về kỹ thuật & dữ liệu kiểm thử

- Khó khăn trong việc xác thực các chỉ số hiệu suất phức tạp
 - Khó đánh giá thời gian phản hồi, thông lượng, tài nguyên hệ thống
 - Thiếu tiêu chí định lượng rõ ràng
- Chuẩn bị test mất nhiều thời gian
 - Tạo dữ liệu, viết script, xây dựng scenario thủ công, dễ lỗi



II. AI TRONG KIỂM THỬ HIỆU NĂNG

1. Thách Thức Trong Kiểm Thử Hiệu Suất

Hạn chế trong phân tích & đánh giá kết quả

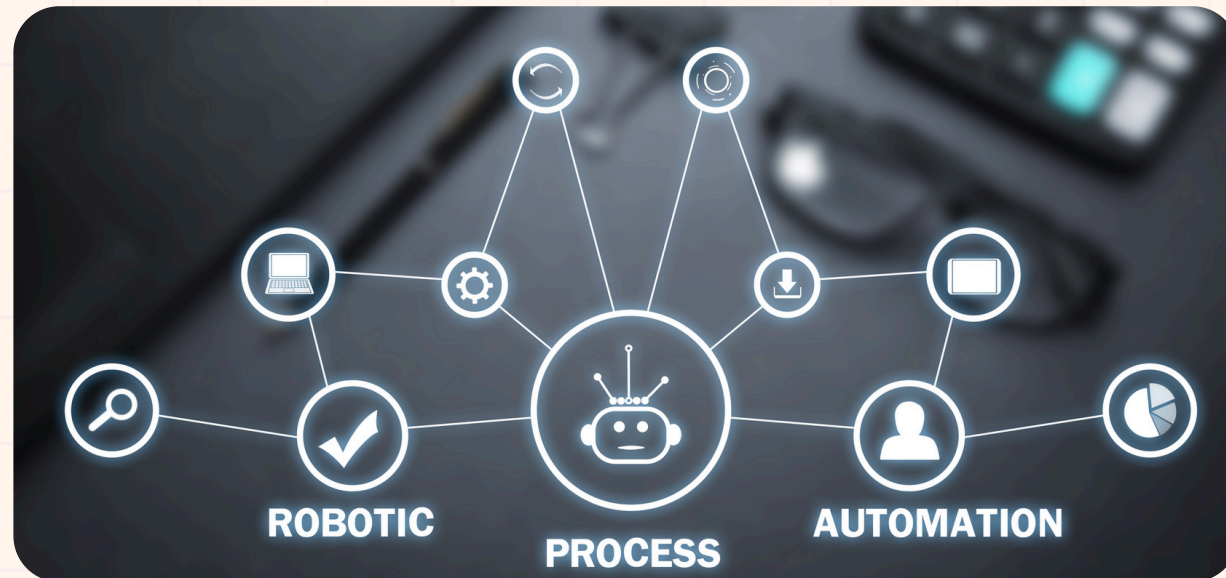
- Khó xác định bottleneck hoặc lỗi ẩn
 - Cần có kinh nghiệm để phân tích log, biểu đồ hiệu suất
 - Lỗi khó tái hiện, đặc biệt ở tải lớn hoặc sau thời gian dài
- Thiếu công cụ giám sát thông minh
 - Công cụ truyền thống chỉ thu thập số liệu thô, thiếu insight



II. AI TRONG KIỂM THỬ HIỆU NĂNG

2. Giải Pháp Đang Có – Liệu Đã Đủ?

Chưa đủ → Đó là lúc AI xuất hiện để hỗ trợ sâu hơn.



01

Tự động hóa quy trình

- Sinh test case & test script tự động
- Rút ngắn thời gian, giảm công sức thủ công
- Tăng tốc độ kiểm thử và mở rộng độ bao phủ kịch bản



02

Phân tích thông minh

- Tự động phân tích dữ liệu kiểm thử ở quy mô lớn
- Nhận diện mẫu lưu lượng và bất thường trong hành vi hệ thống
- Gợi ý điều chỉnh và tối ưu theo thời gian thực

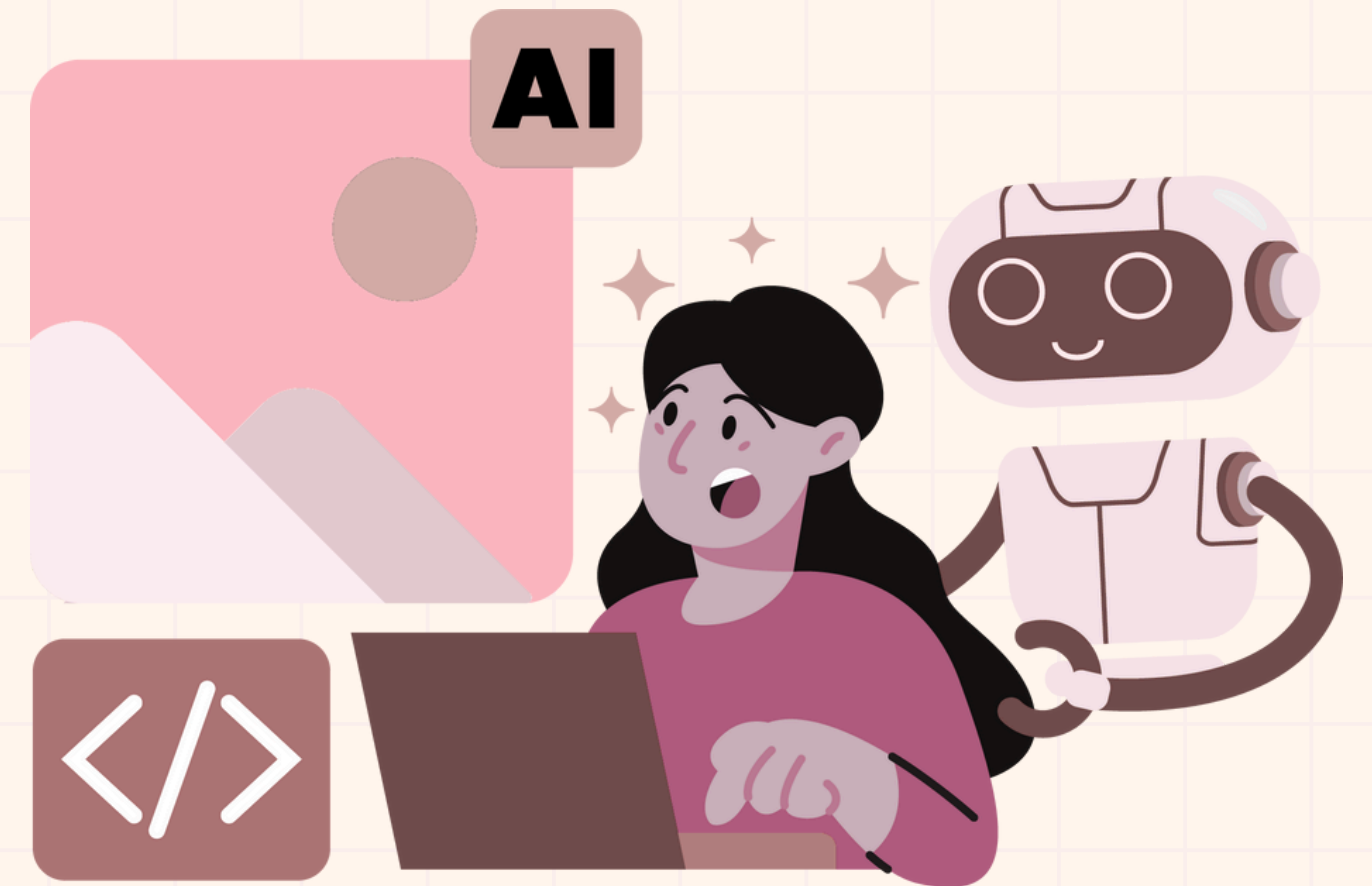
II. AI TRONG KIỂM THỬ HIỆU NĂNG

3. AI Mang Lại Những Lợi Thế Gì?

AI không chỉ giúp “test nhanh hơn” – mà còn giúp hệ thống trở nên thông minh hơn trong cách đánh giá và tối ưu.

Tự động hóa việc tạo test

- Phân tích production traffic → tạo hàng ngàn test scenario thực tế
- Bao phủ edge case mà test thủ công dễ bỏ sót



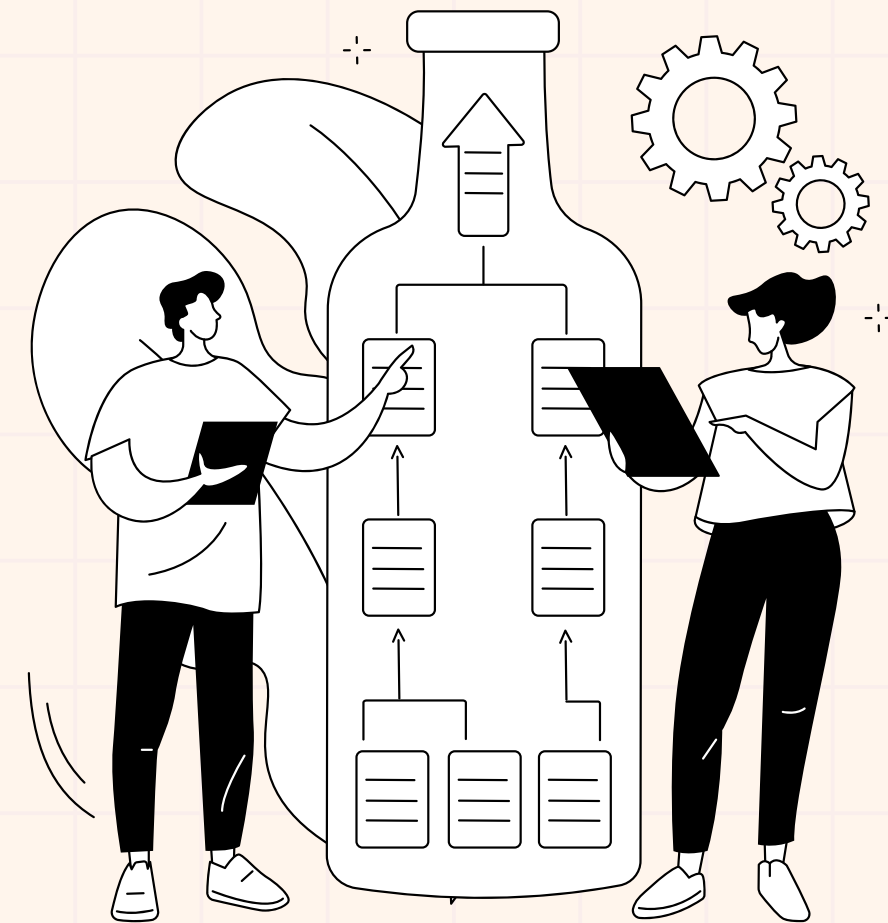
II. AI TRONG KIỂM THỬ HIỆU NĂNG

3. AI Mang Lại Những Lợi Thế Gì?

AI không chỉ giúp “test nhanh hơn” – mà còn giúp hệ thống trở nên thông minh hơn trong cách phản ứng và tối ưu.

Phát hiện bottleneck nhanh chóng

- Tự động hóa toàn bộ quá trình phân tích
- Học từ dữ liệu lịch sử và mẫu bất thường kết hợp với phân tích đa chiều
- Không cần chờ người vận hành can thiệp



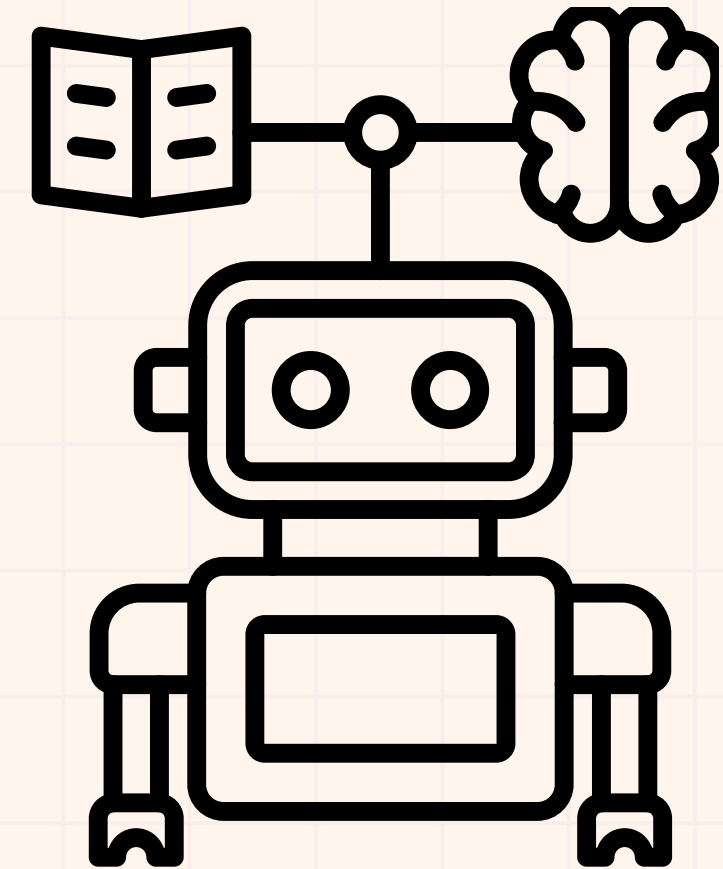
II. AI TRONG KIỂM THỬ HIỆU NĂNG

3. AI Mang Lại Những Lợi Thế Gì?

AI không chỉ giúp “test nhanh hơn” – mà còn giúp hệ thống trở nên thông minh hơn trong cách phản ứng và tối ưu.

Dự đoán hiệu suất với Machine Learning

- Phân tích dữ liệu hiệu suất trong quá khứ bằng ML
- Dự đoán sớm traffic peak (VD: Black Friday)
- Ước lượng chính xác nhu cầu tài nguyên
- Hỗ trợ lập kế hoạch mở rộng hệ thống hiệu quả

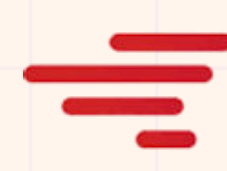


II. AI TRONG KIỂM THỬ HIỆU NĂNG

4. Tổng Quan Các Nền Tảng Kiểm Thử Hiệu Năng Có Hỗ Trợ AI



dynatrace

 **BlazeMeter**



II. AI TRONG KIỂM THỬ HIỆU NĂNG

4. Tổng Quan Các Nền Tảng Kiểm Thử Hiệu Năng Có Hỗ Trợ AI

Công cụ	Mô tả	Nhược điểm chính
Dynatrace	Tự động phát hiện bất thường & phân tích nguyên nhân gốc (root cause) bằng AI (Davis AI) từ logs, traces, metrics.	Giá cao; cài đặt và cấu hình phức tạp
BlazeMeter	Hỗ trợ kiểm thử tải/áp lực quy mô lớn; phân tích dữ liệu & gợi ý tối ưu script/test data.	Cần chuyên môn kỹ thuật để sử dụng hiệu quả
LambdaTest (HyperExecute)	Cho phép chạy các load tests từ cloud (ví dụ JMeter), thu thập KPI như response time và error insights trên nhiều vùng và môi trường thử nghiệm	Không có khả năng phân tích AI nâng cao root-cause
Postman	Thực hiện API performance testing, mô phỏng tải với Virtual Users, theo dõi response time, error rate, throughput.	Khả năng kiểm thử hiệu năng quy mô rất lớn/phức tạp còn hạn chế so với công cụ chuyên dụng.

II. AI TRONG KIỂM THỬ HIỆU NĂNG

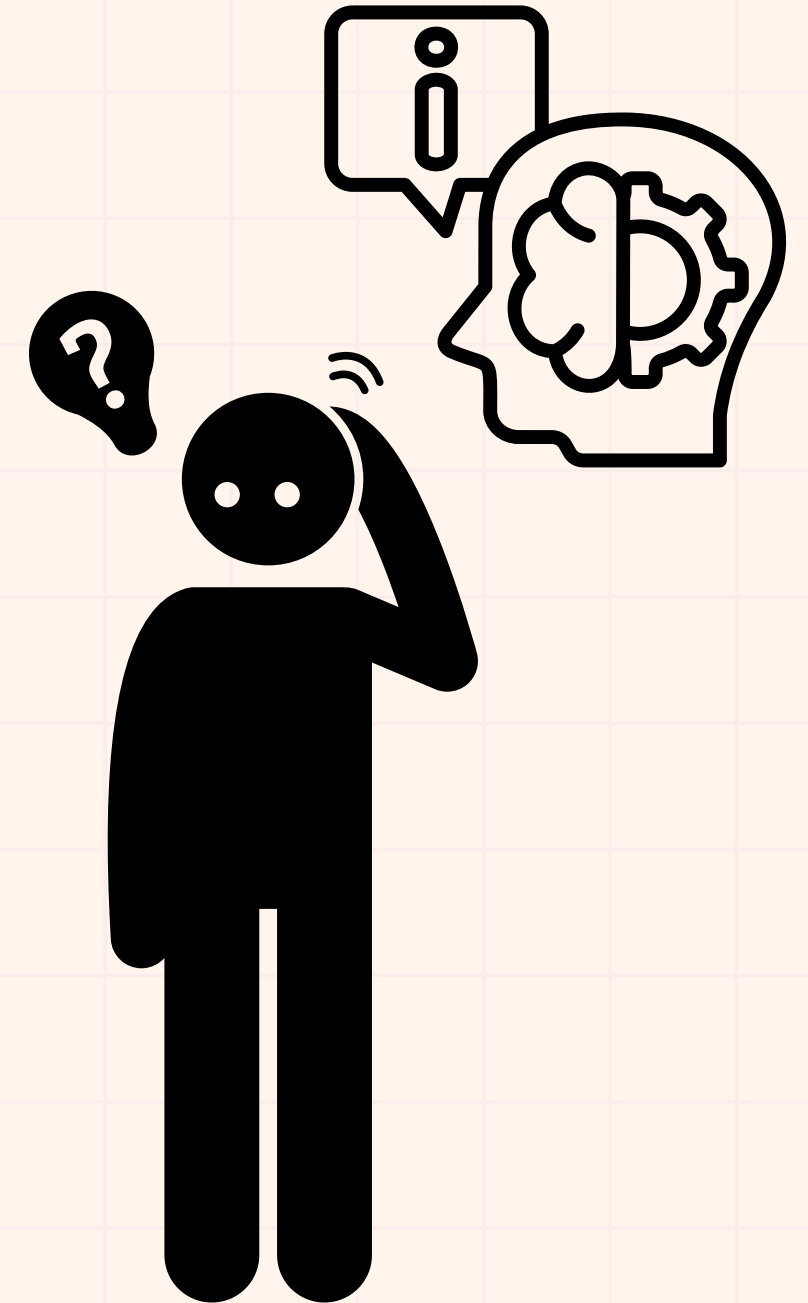
5. Hạn Chế Khi Ứng Dụng AI Vào Performance Testing

Prompt sai → Kết quả sai lệch (GIGO)

- Prompt không rõ ràng hoặc thiếu ngữ cảnh dễ khiến AI hiểu sai mục tiêu.
- Thiếu hiểu biết domain → test case không thực tế.

Chi phí đầu tư ban đầu cao

- Máy chủ mạnh, GPU, bản quyền phần mềm AI rất đắt đỏ.
- Cần chuyên gia AI để setup và tinh chỉnh.



II. AI TRONG KIỂM THỬ HIỆU NĂNG

5. Hạn Chế Khi Ứng Dụng AI Vào Performance Testing

Vấn đề “Black Box”

- AI khuyến nghị “tăng RAM 40%” → nhưng không giải thích lý do.
- Khó xác thực, dễ dẫn đến quá phụ thuộc

Phụ thuộc chất lượng dữ liệu

- AI chỉ chính xác nếu dữ liệu đầu vào chất lượng.
- Thiếu dữ liệu lịch sử hoặc dữ liệu lệch → AI dự đoán sai.
- Hậu quả: test không sát thực tế.



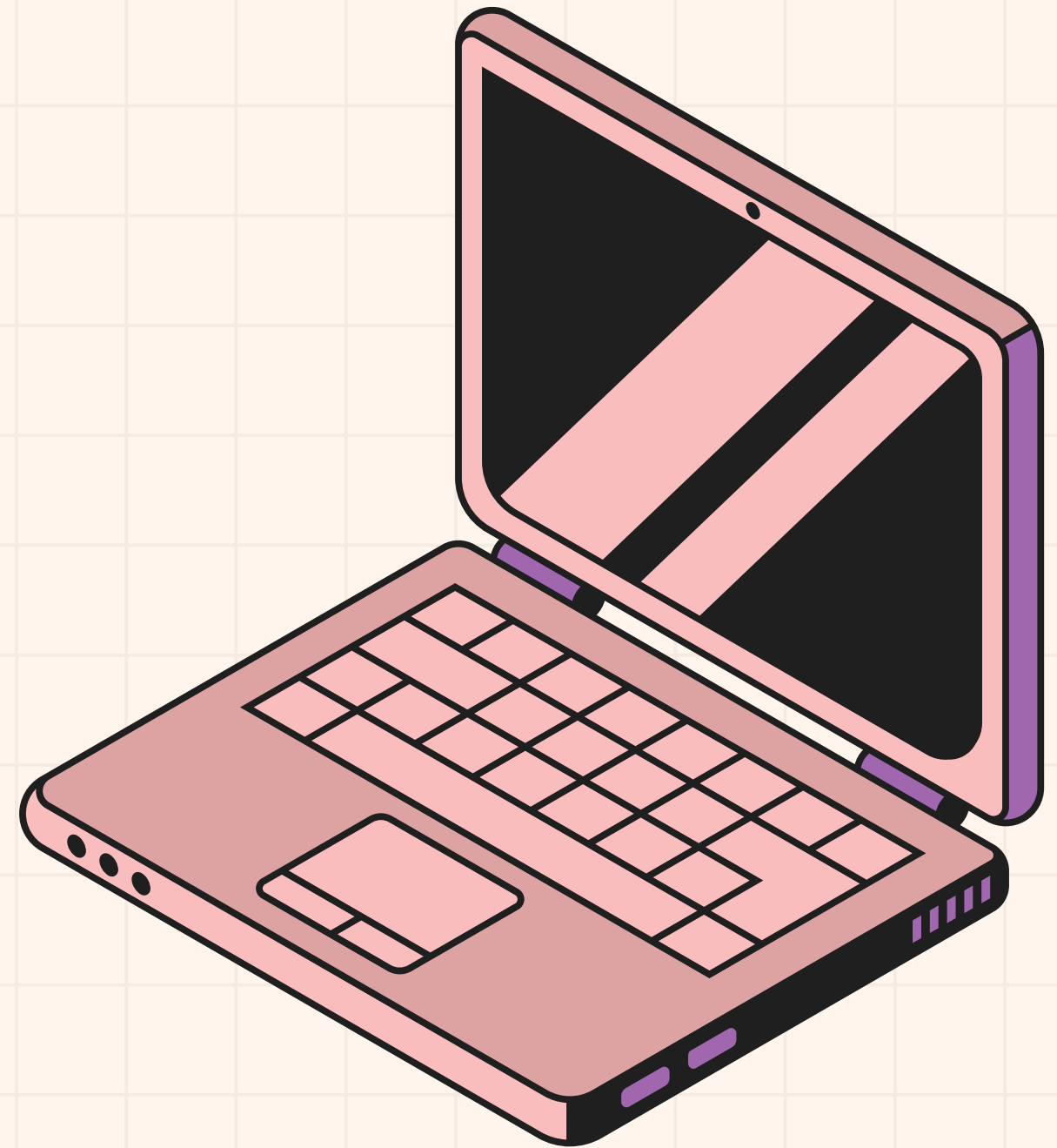
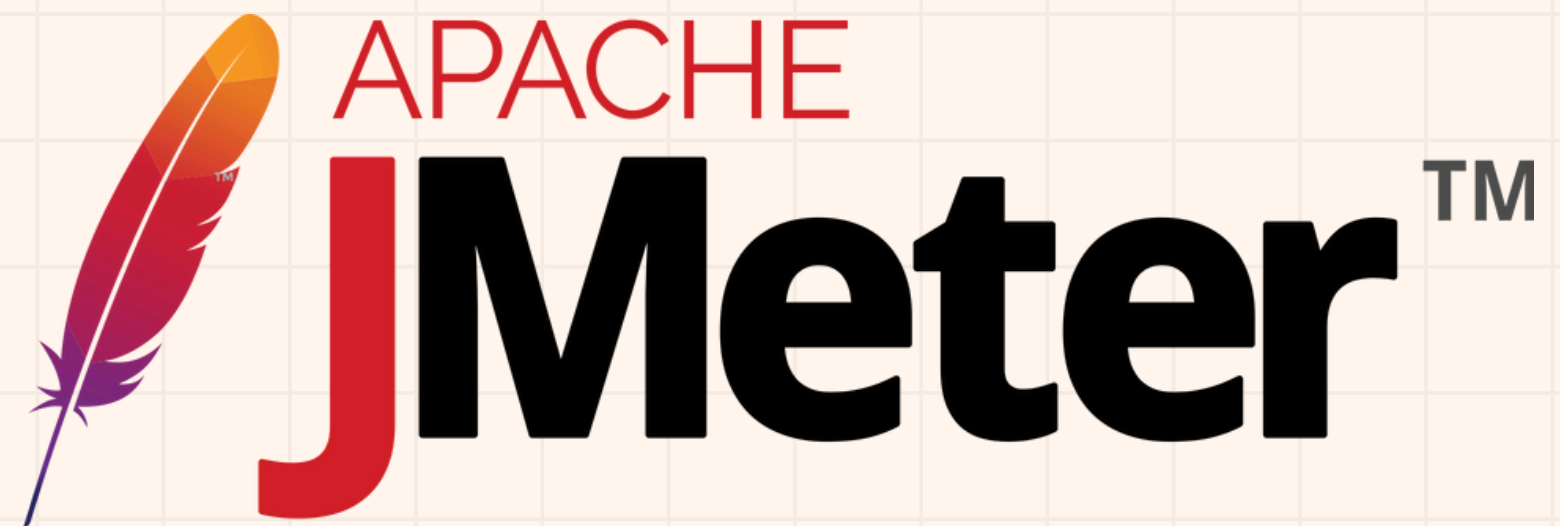
[GIỚI THIỆU](#)

[LÝ THUYẾT](#)

[JMETER](#)

[DEMO](#)

CÔNG CỤ JMETER



III. CÔNG CỤ JMETER

1. Giới thiệu tổng quan



Tổng quát

JMeter là một công cụ kiểm thử hiệu suất mã nguồn mở được phát triển bởi Apache, chủ yếu được sử dụng để kiểm tra độ bền, hiệu suất và khả năng chịu tải của các ứng dụng web, API và máy chủ.

Tiện ích

Công cụ hỗ trợ nhiều giao thức như HTTP, FTP, JDBC và SOAP, đồng thời có khả năng mở rộng thông qua các plugin.

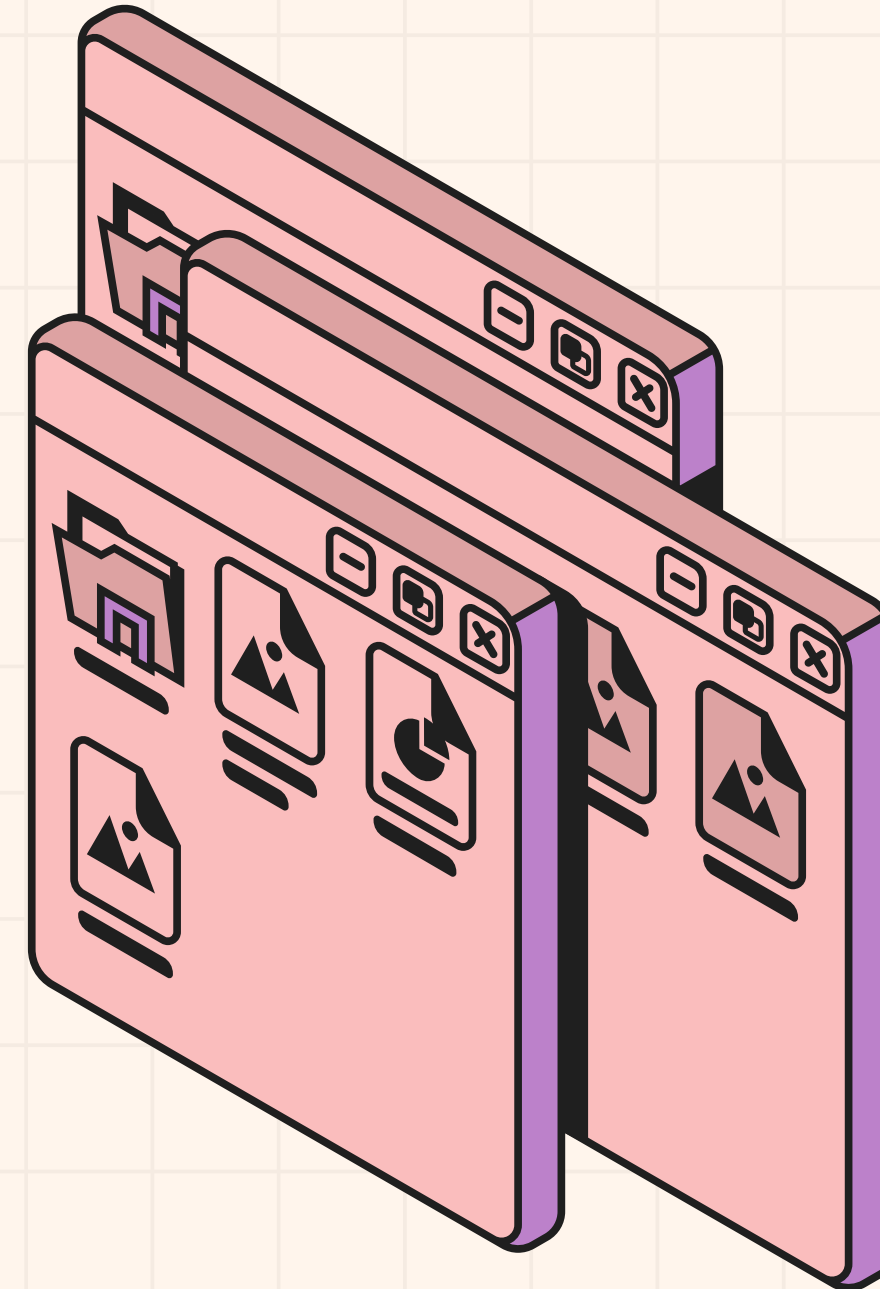
Công cụ này được viết bằng Java, có thể chạy trên Windows, macOS, Linux và hỗ trợ cả giao diện đồ họa (GUI) lẫn dòng lệnh (CLI).

III. CÔNG CỤ JMETER

2. Cài đặt công cụ

Yêu cầu hệ thống

- JMeter yêu cầu Java (JDK) phiên bản 8 trở lên, tương thích tốt nhất với Java 17.
- Hệ điều hành hỗ trợ: Windows, Linux, macOS
- Không cần cài đặt thêm môi trường nào khác, chỉ cần giải nén và chạy.



III. CÔNG CỤ JMETER

2. Cài đặt công cụ

Tải và cài đặt JMeter

Truy cập trang tải: https://jmeter.apache.org/download_jmeter.cgi



About

- Overview
- License

Download

- Download Releases
- Release Notes

Documentation

- Get Started
- User Manual
- Best Practices
- Component Reference
- Functions Reference
- Properties Reference
- Change History
- Javadocs
- JMeter Wiki
- FAQ (Wiki)

Tutorials

- Distributed Testing
- Recording Tests
- JUnit Sampler
- Access Log Sampler
- Extending JMeter

Community

- Issue Tracking
- Security
- Mailing Lists
- Source Repositories
- Building and Contributing
- Project info at Apache

Download Apache JMeter

[Twitter](#)[github](#)

We recommend you use a mirror to download our release builds, but you **must** [verify the integrity](#) of the downloaded files using signatures downloaded from our main distribution directories. Recent releases (48 hours) may not yet be available from all the mirrors.

You are currently using **<https://d1cdn.apache.org/>**. If you encounter a problem with this mirror, please select another mirror. If all mirrors are failing, there are *backup* mirrors (at the end of the mirrors list) that should be available.

Other mirrors:

The **KEYS** link links to the code signing keys used to sign the product. The **PGP** link downloads the OpenPGP compatible signature from our main site. The **SHA-512** link downloads the sha512 checksum from the main site. Please [verify the integrity](#) of the downloaded file.

For more information concerning Apache JMeter, see the [Apache JMeter](#) site.

[KEYS](#)

Apache JMeter 5.6.3 (Requires Java 8+)

Binaries

[apache-jmeter-5.6.3.tgz sha512 pgp](#)
[apache-jmeter-5.6.3.zip sha512 pgp](#)

Source

III. CÔNG CỤ JMETER

2. Cài đặt công cụ

Tải và cài đặt JMeter

- Tải file nén .zip (Windows) hoặc .tgz (Linux/macOS)
- Giải nén vào một thư mục bất kỳ
- Không cần cài đặt – chỉ cần chạy file .bat hoặc .sh trong thư mục bin

Apache JMeter 5.6.3 (Requires Java 8+)

Binaries

[apache-jmeter-5.6.3.tgz sha512 pgp](#)
[apache-jmeter-5.6.3.zip sha512 pgp](#)

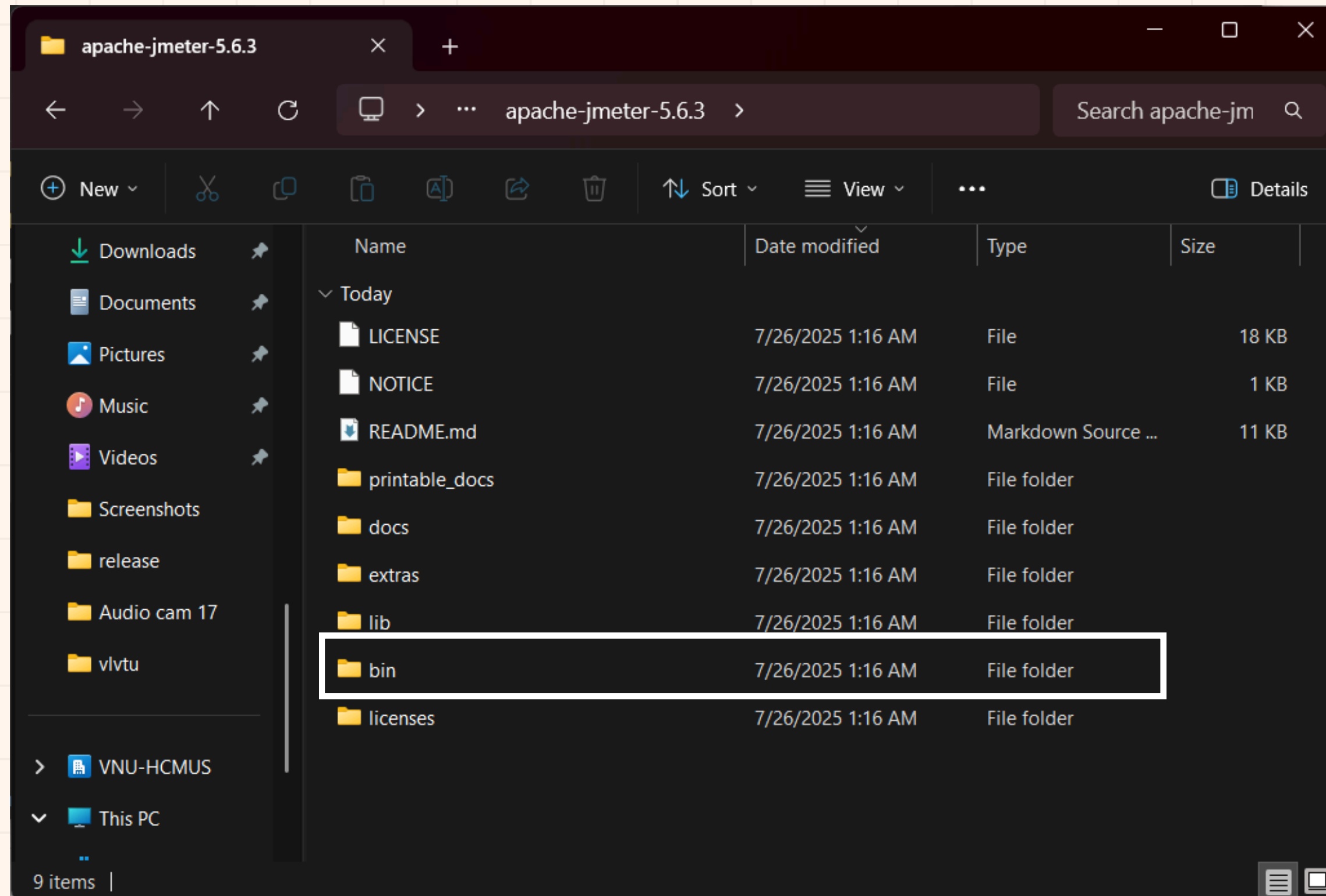


Source

[apache-jmeter-5.6.3_src.tgz sha512 pgp](#)
[apache-jmeter-5.6.3_src.zip sha512 pgp](#)

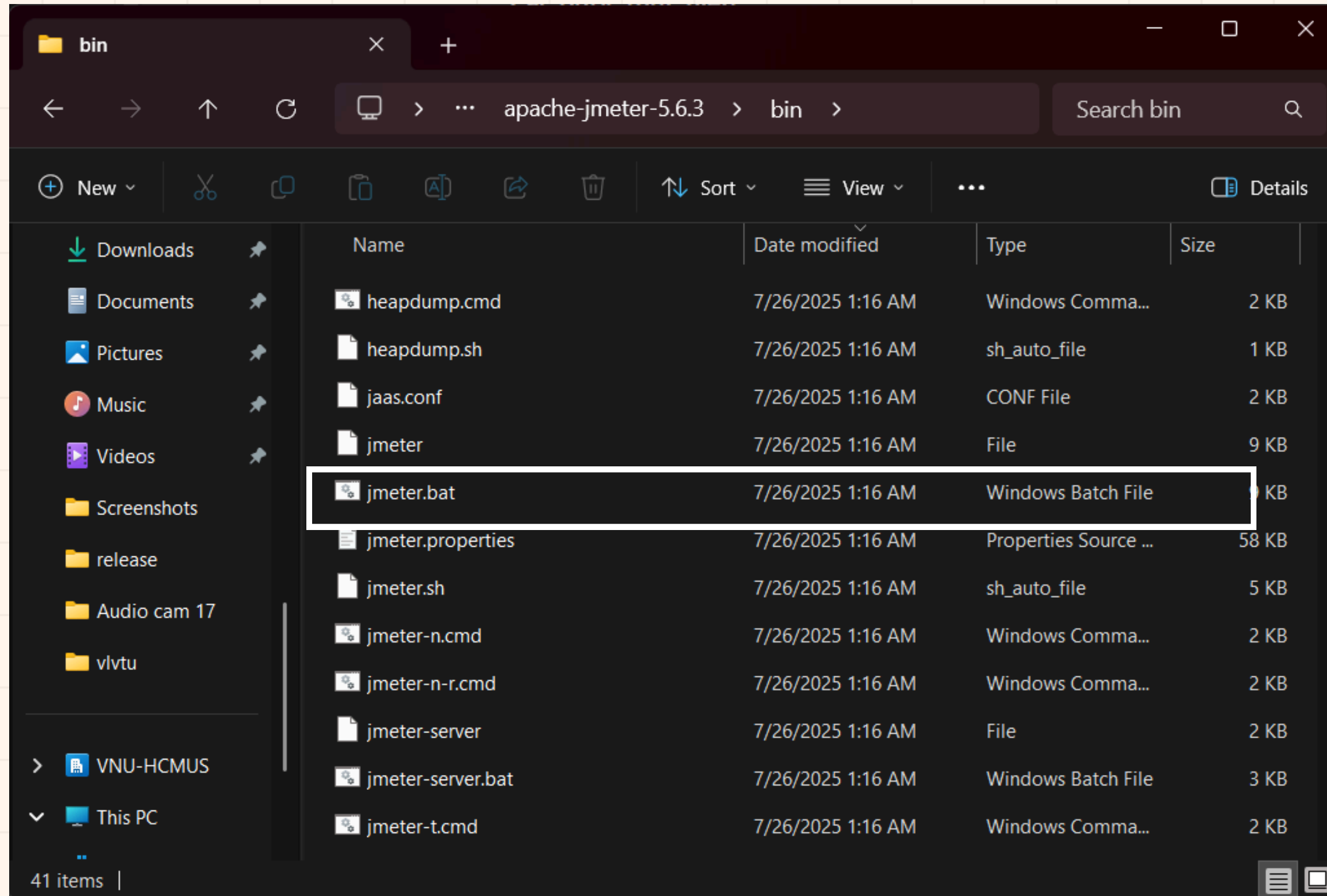
III. CÔNG CỤ JMETER

3. Khởi động JMeter



III. CÔNG CỤ JMETER

3. Khởi động JMeter

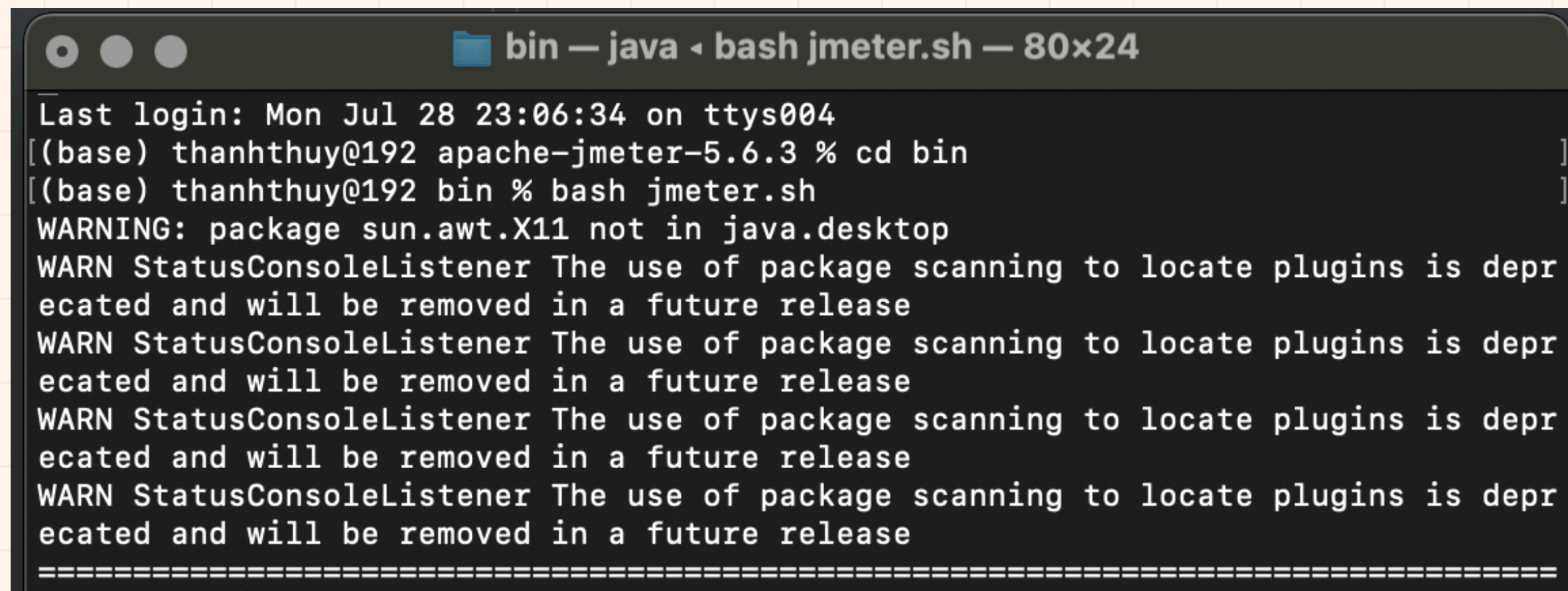


III. CÔNG CỤ JMETER

3. Khởi động JMeter

Đối với Linux và MacOS:

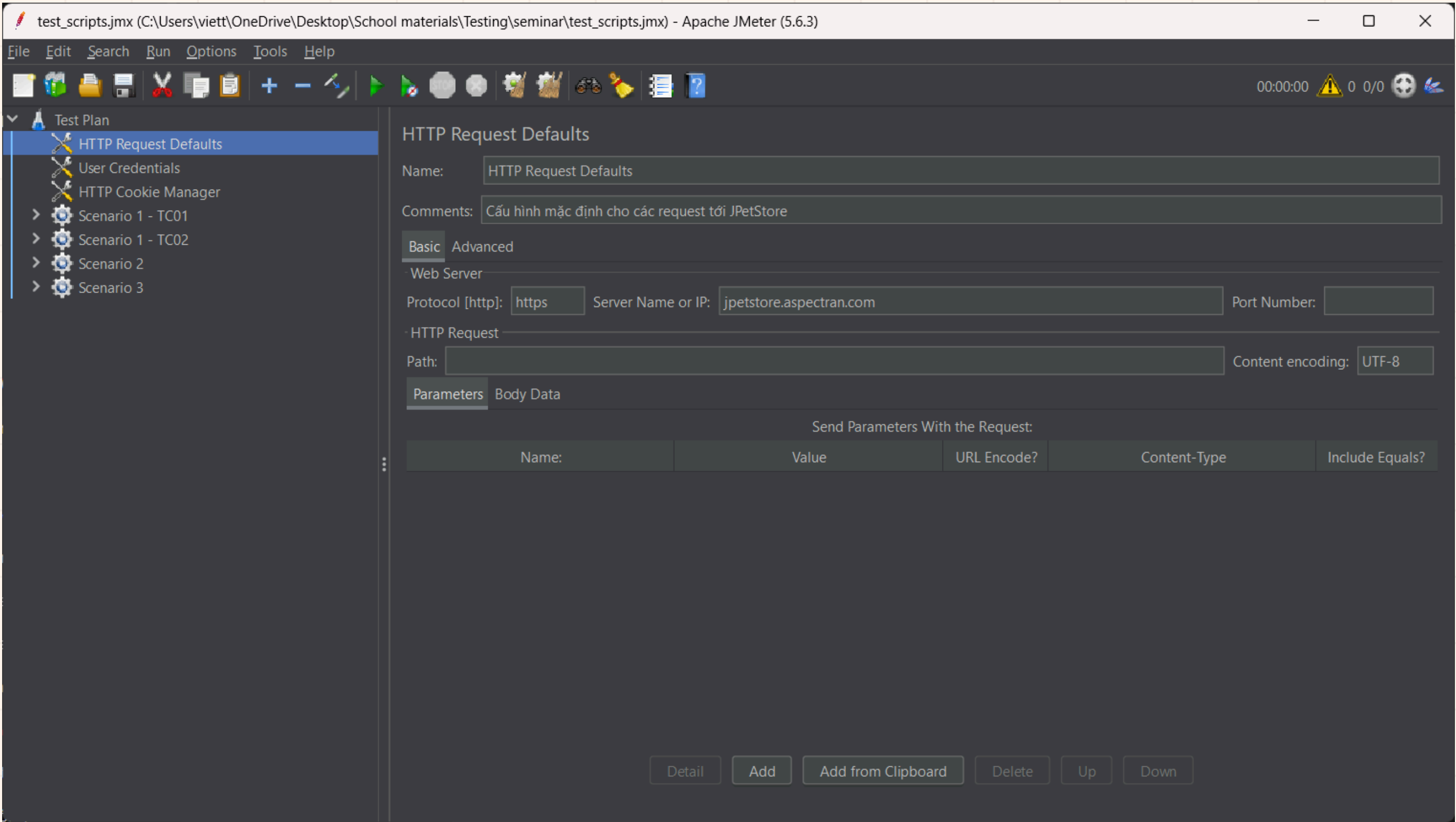
- Mở Terminal
- Di chuyển đến folder “**apache-jmeter-5.6.3\bin**”
- Chạy lệnh **`bash jmeter.sh`**

A screenshot of a macOS Terminal window. The title bar shows 'bin — java ▾ bash jmeter.sh — 80x24'. The terminal content shows the user logging in, navigating to the 'bin' directory, and running 'bash jmeter.sh'. This triggers several deprecation warnings from the 'StatusConsoleListener' about package scanning for plugins. The output ends with a line of 25 equals signs.

```
bin — java ▾ bash jmeter.sh — 80x24
Last login: Mon Jul 28 23:06:34 on ttys004
(base) thanhthuy@192 apache-jmeter-5.6.3 % cd bin
(base) thanhthuy@192 bin % bash jmeter.sh
WARNING: package sun.awt.X11 not in java.desktop
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
=====
```

III. CÔNG CỤ JMeter

3. Khởi động JMeter



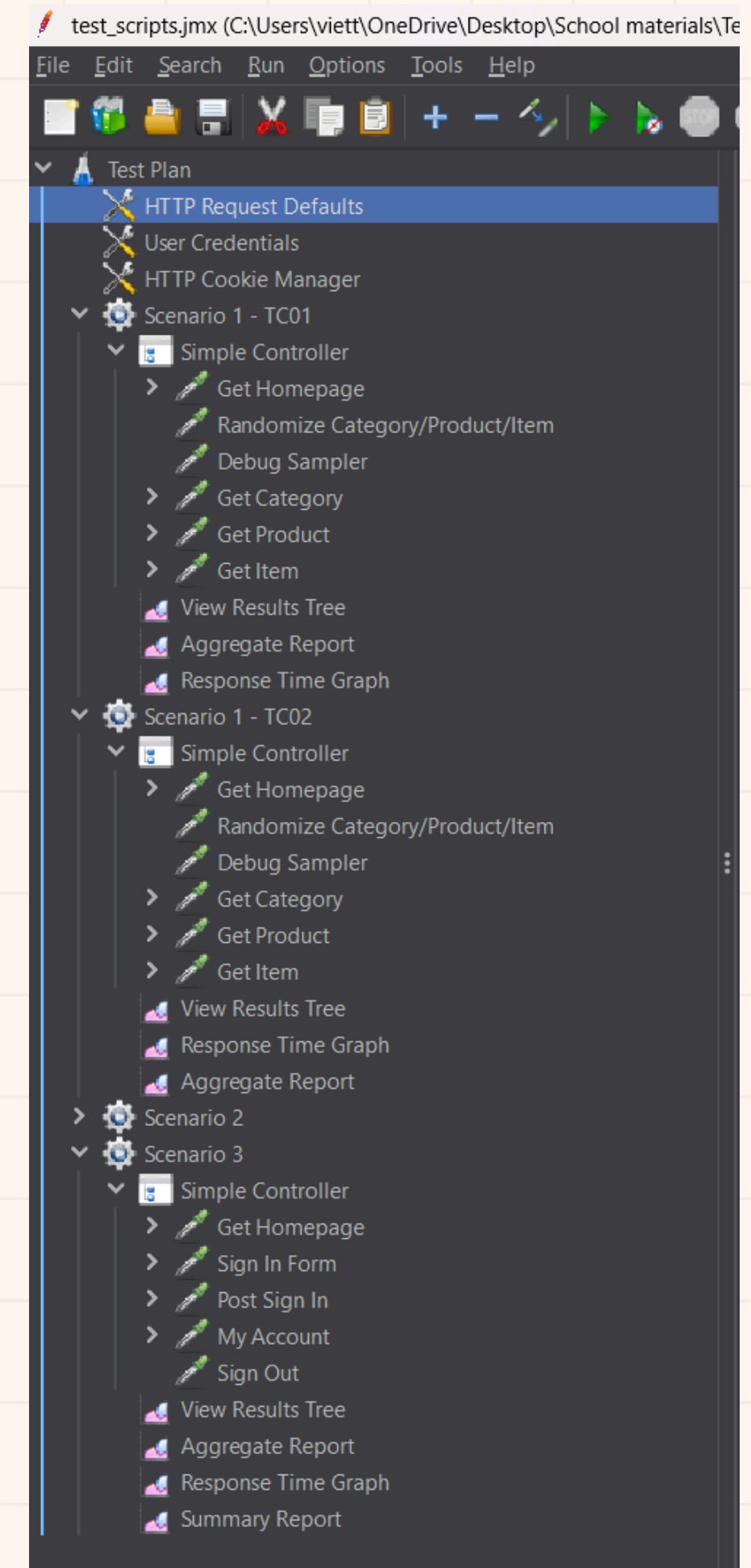
III. CÔNG CỤ JMETER

4. Các thành phần chính

Test plan: Là “khung” chính bao quát toàn bộ kịch bản kiểm thử, chứa tất cả các Thread Group, Config Elements, Samplers, Listeners...

Chức năng:

- Quy định tên, mô tả chung về mục tiêu của bài kiểm thử.
- Cho phép cấu hình biến toàn cục (User Defined Variables), thiết lập logic điều khiển (Test Fragment, Module Controller).
- Hỗ trợ phép lặp lồng nhau, điều kiện (If Controller), vòng lặp có điều kiện (While Controller).



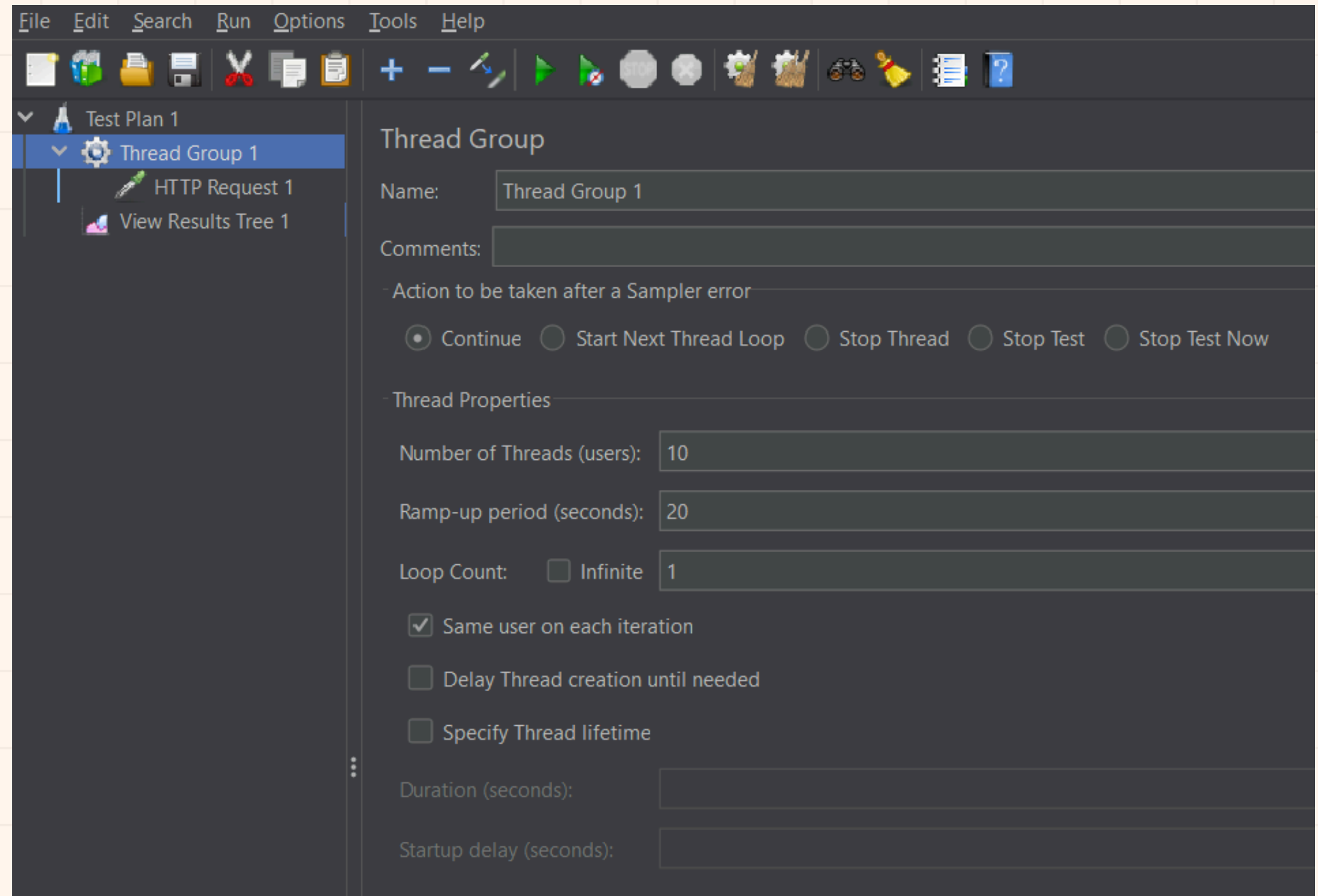
III. CÔNG CỤ JMETER

4. Các thành phần chính

Thread group: Thành phần mô phỏng “người dùng ảo” thực hiện song song các hành động trong kịch bản. Giả lập tải thấp cao khác nhau để đo khả năng chịu tải của ứng dụng.

Các tham số chính:

- Number of Threads (users): số lượng user ảo đồng thời.
- Ramp-Up Period (seconds): thời gian tăng dần để khởi tạo hết tất cả các thread.
- Loop Count: số lần mỗi thread lặp lại kịch bản; có thể chọn “Forever” để chạy liên tục.



III. CÔNG CỤ JMETER

4. Các thành phần chính

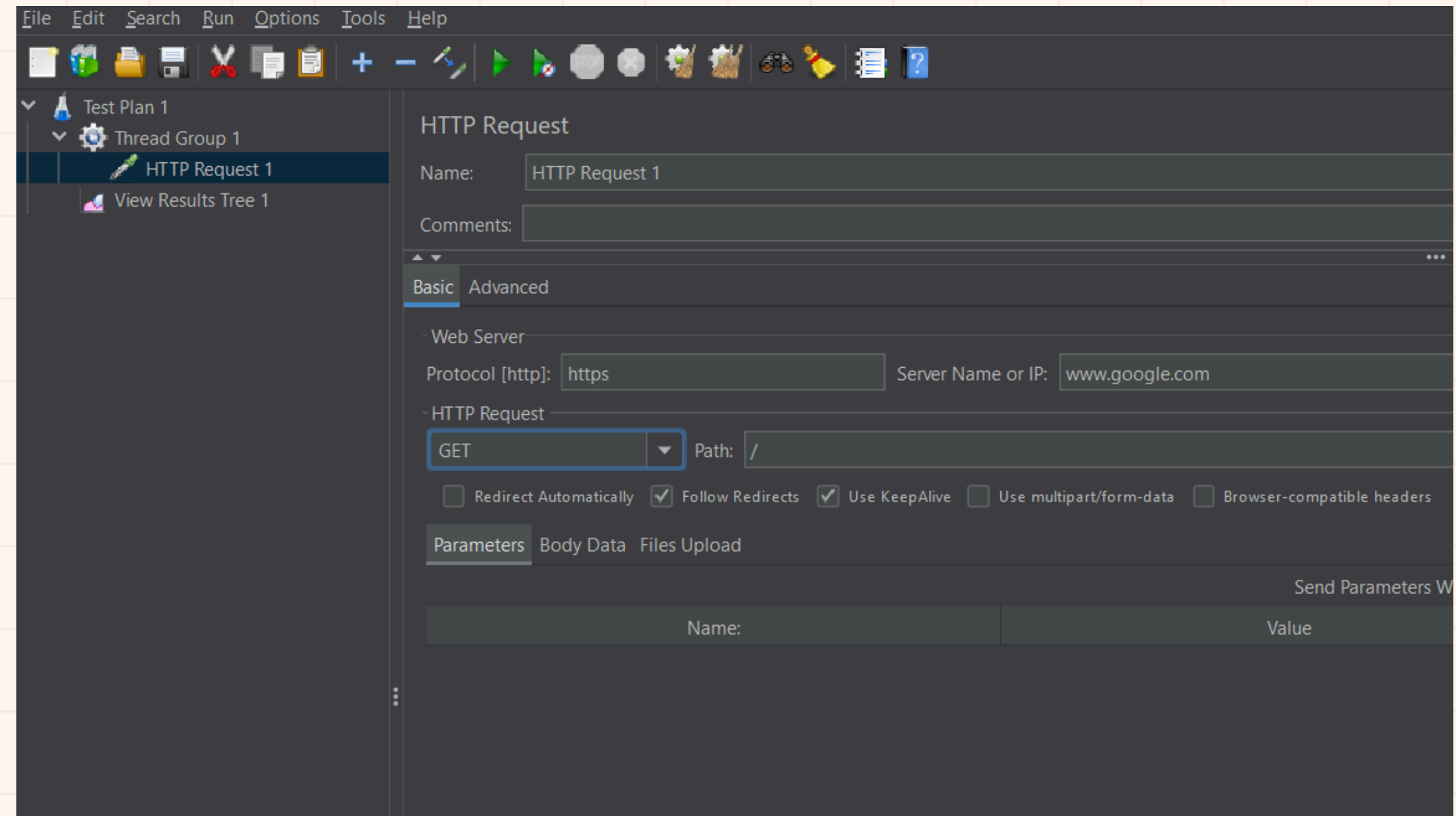
Sampler: Là nơi cấu hình các request mà user ảo sẽ gửi đi.

Các loại thông dụng:

- HTTP Request: GET/POST, thiết lập header, parameters, body.
- JDBC Request: thực thi SQL query đến database.
- SOAP/XML-RPC Request & REST Request: test API SOAP/REST.

Thiết lập chính:

- Địa chỉ URL hoặc Data Source, phương thức (GET/POST), tham số, header, authentication.



III. CÔNG CỤ JMETER

4. Các thành phần chính

Listener: Thành phần thu thập, hiển thị và xuất kết quả test.

Một số Listener tiêu biểu:

- View Results Tree: xem chi tiết từng request/response.
- Aggregate Report: thống kê nhanh.
- Summary Report: bản tóm tắt gọn nhẹ, tiết kiệm bộ nhớ.
- Graph Results / Response Time Graph: biểu đồ trực quan theo thời gian.

ol materials\Testing\seminar\test_scripts.jmx) - Apache JMeter (5.6.3)

03:31:39 0 0/450

Aggregate Report

Name: Aggregate Report

Comments:

Write results to file / Read from file

Filename Browse... Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughp...	Received ...	Sent KB/s...
Get Hom...	302	194	153	435	458	474	142	558	0.00%	1.0/sec	23.93	0.17
Sign In Fo...	300	167	151	161	237	458	142	480	0.00%	1.0/sec	22.86	0.20
Post Sign ...	292	307	299	314	348	582	280	598	0.00%	1.0/sec	23.26	0.49
Randomi...	292	1	1	3	3	3	0	4	0.00%	1.0/sec	0.00	0.00
Debug Sa...	292	0	0	0	0	1	0	1	0.00%	1.0/sec	0.38	0.00
Get Cate...	286	173	154	179	372	453	142	473	0.00%	1.0/sec	22.69	0.20
Get Prod...	277	196	157	438	461	479	144	488	0.00%	1.0/sec	22.63	0.20
Get Item	269	180	156	231	439	462	144	471	0.00%	1.0/sec	22.43	0.21
Add to C...	269	302	299	312	315	413	283	538	0.00%	1.0/sec	25.11	0.70
View Cart	268	152	151	157	160	164	142	386	0.00%	1.0/sec	24.55	0.32
Proceed t...	268	300	298	311	314	383	281	519	0.00%	1.0/sec	23.13	0.42
Continue ...	268	298	298	310	312	319	281	497	0.00%	1.0/sec	23.08	0.50
Confirm ...	268	298	298	310	312	345	281	403	0.00%	1.0/sec	23.08	0.50
TOTAL	3651	195	158	308	316	466	0	598	0.00%	12.3/sec	238.23	3.59

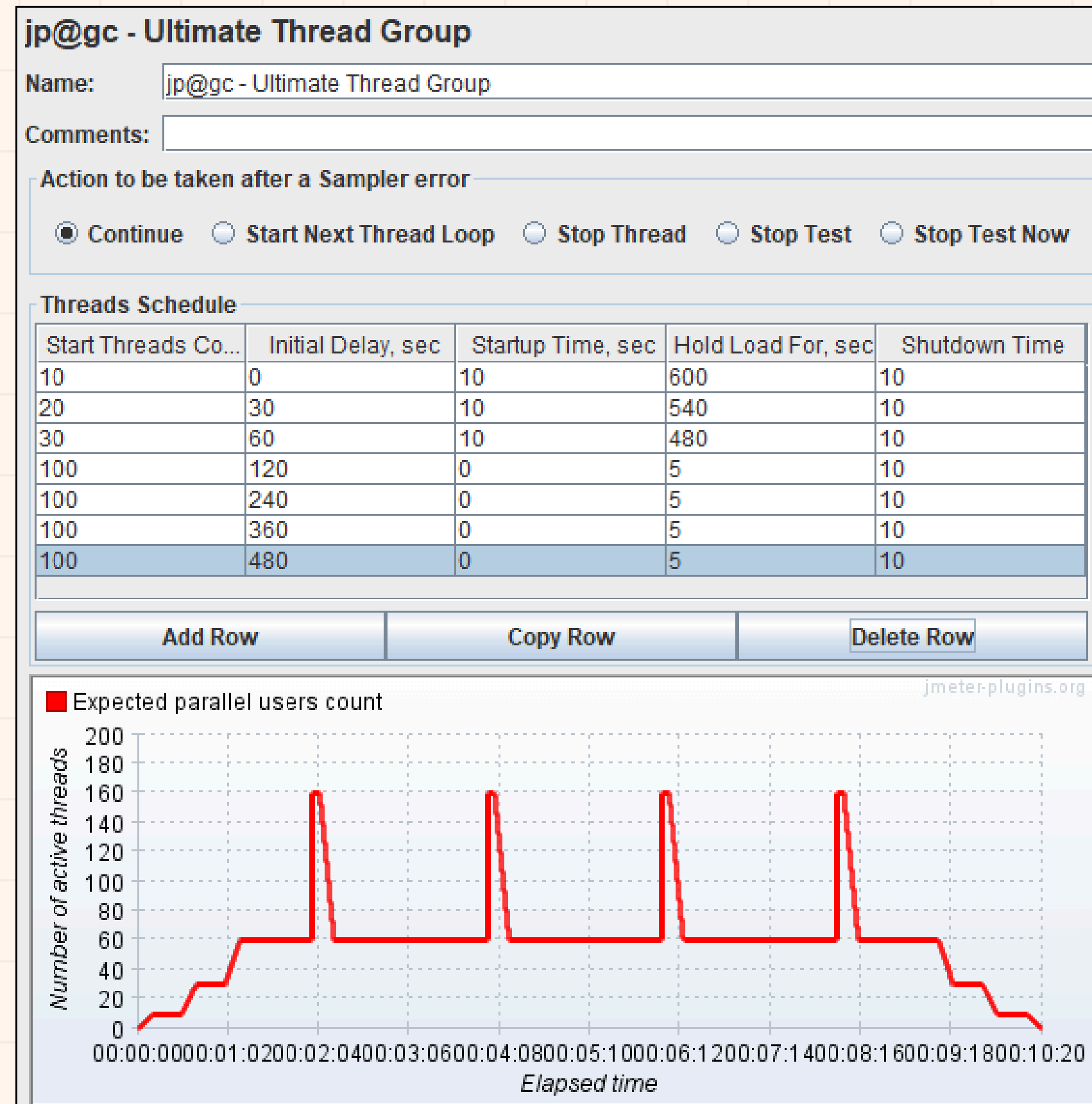
☐ Include group name in label? ☒ Save Table Header

III. CÔNG CỤ JMETER

5. Các Plugins Nâng Cao

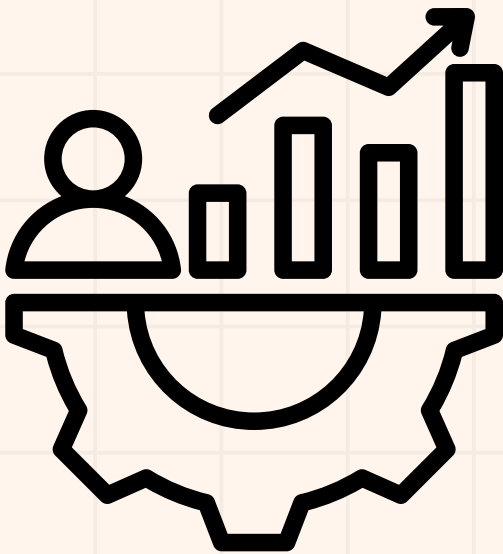
Ultimate Thread Group

- Tùy chỉnh số lượng người dùng (threads) theo từng thời điểm.
- Xác định rõ thời gian bắt đầu (delay), thời gian tăng tải (ramp-up), thời gian giữ tải (hold load), và thời gian giảm tải (shutdown).
- Thiết lập nhiều “giai đoạn” khác nhau cho việc mô phỏng tải (multi-stage load test).



III. CÔNG CỤ JMETER

5. Các Plugins Nâng Cao



Cấu hình một dòng trong Ultimate Thread Group

- **Start Threads = 5**: sẽ tạo 5 người dùng (threads).
- **Initial Delay = 0s**: không chờ, bắt đầu ngay lập tức.
- **Startup Time = 25s**: 5 người dùng sẽ được khởi tạo từ từ trong 25 giây.
- **Hold Load For = 300s**: giữ tải ổn định (5 người dùng hoạt động) trong 5 phút.
- **Shutdown Time = 50s**: 5 người dùng này sẽ được dừng lại từ từ trong vòng 50 giây.

Start Thread	Initial Delay	Startup Time	Hold Load For	Shutdown Time
5	0	25	300	50

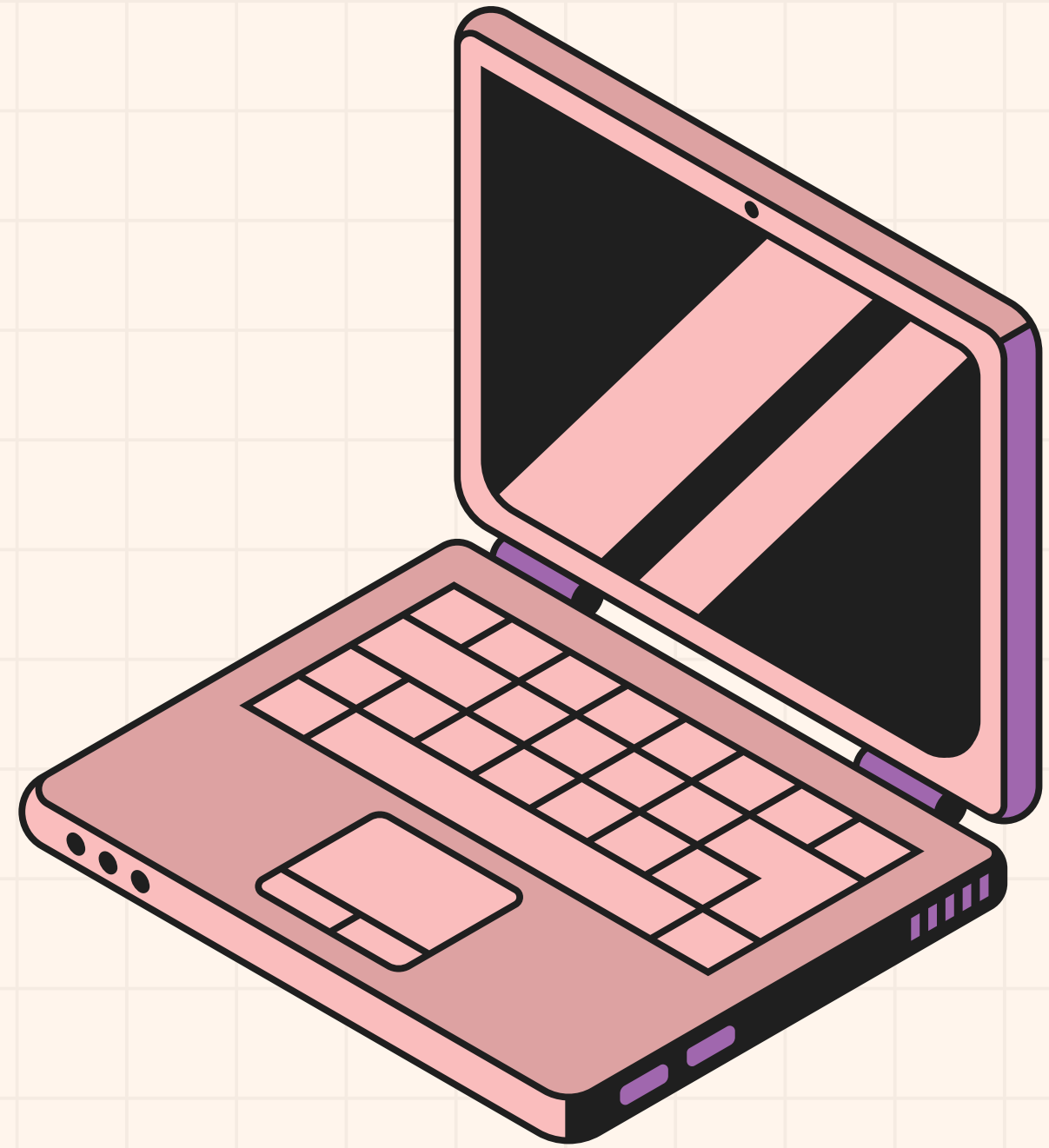
GIỚI THIỆU

LÝ THUYẾT

JMETER

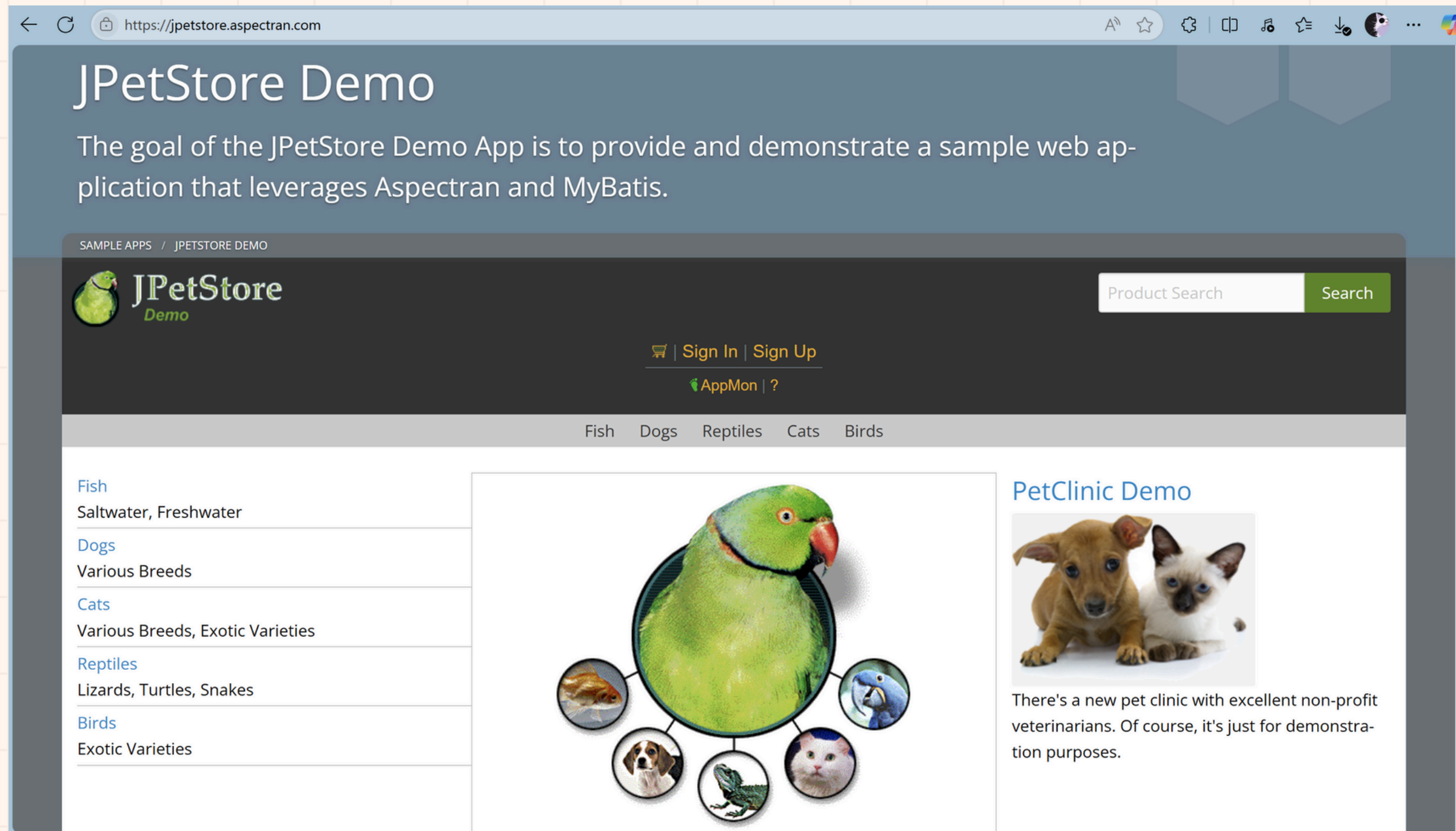
DEMO

DEMO



IV. DEMO

1. Đối tượng kiểm thử

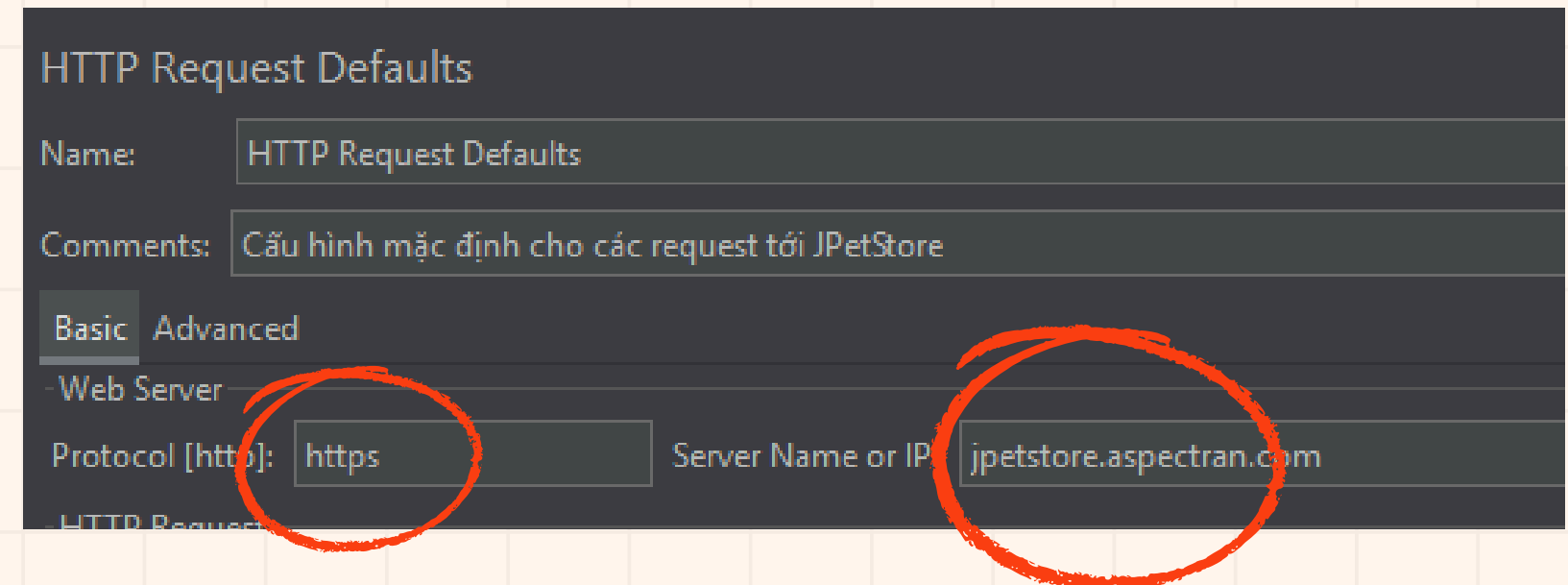


<https://jpetstore.aspectran.com>

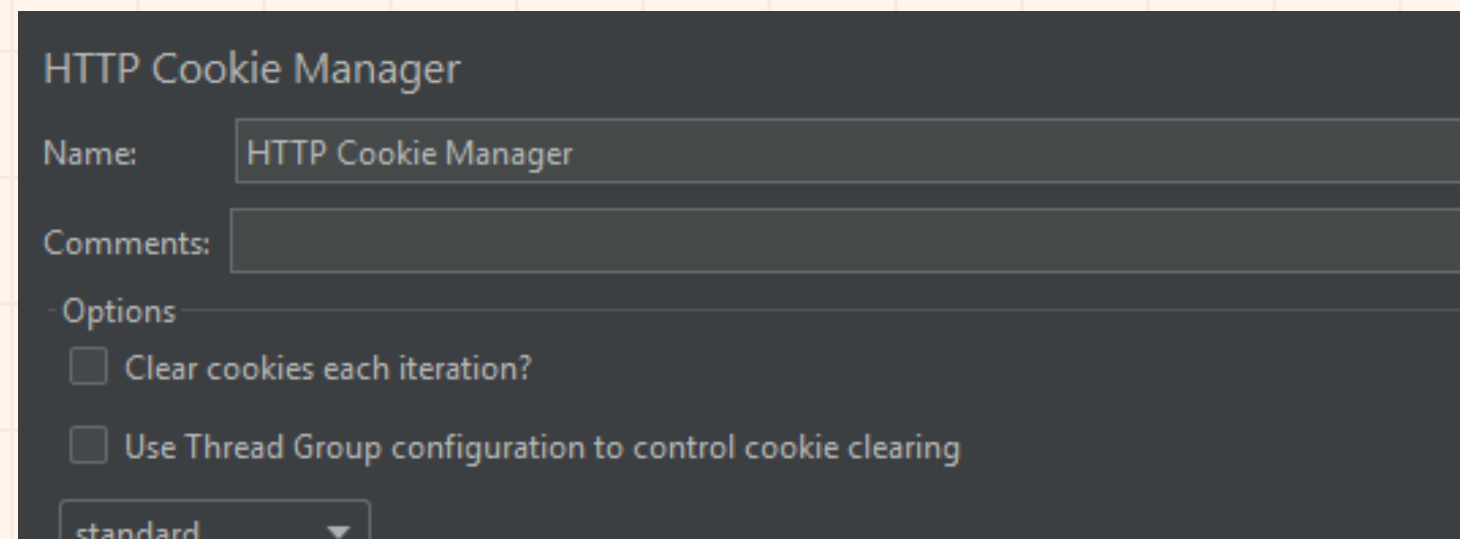
IV. DEMO

2. Cấu hình đầu vào

HTTP Request Defaults: Cấu hình mặc định cho các request tới JPetStore



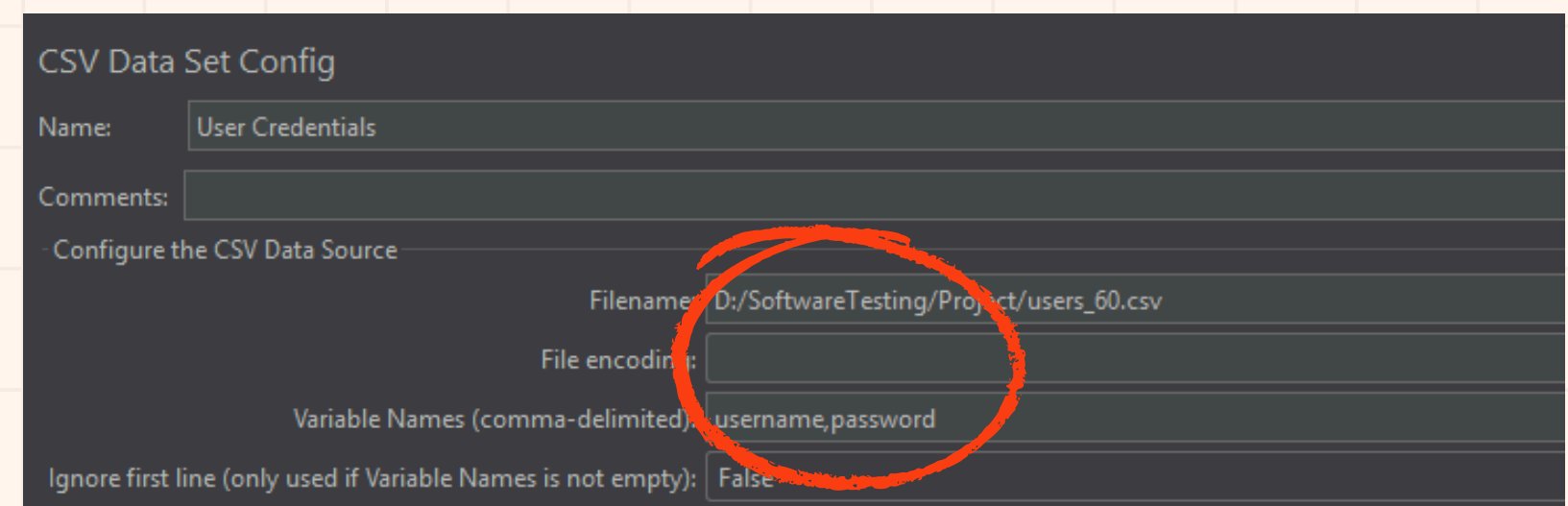
The screenshot shows the 'HTTP Request Defaults' configuration window. The 'Name' field is 'HTTP Request Defaults' and the 'Comments' field is 'Cấu hình mặc định cho các request tới JPetStore'. The 'Basic' tab is selected. The 'Web Server' section shows 'Protocol [http]: https' and 'Server Name or IP: jpetstore.aspectran.com', both of which are circled in red.



The screenshot shows the 'HTTP Cookie Manager' configuration window. The 'Name' field is 'HTTP Cookie Manager'. The 'Options' section has two unchecked checkboxes: 'Clear cookies each iteration?' and 'Use Thread Group configuration to control cookie clearing'. A dropdown menu at the bottom is set to 'standard'.

HTTP Cookie Manager: Quản lý cookie trong quá trình kiểm tra hiệu năng

CSV Data Set Config: Cấu hình các file data sẽ sử dụng



The screenshot shows the 'CSV Data Set Config' configuration window. The 'Name' field is 'User Credentials'. The 'Configure the CSV Data Source' section shows 'Filename: D:/SoftwareTesting/Project/users_60.csv', 'File encoding: ' (empty), and 'Variable Names (comma-delimited): username,password', which is circled in red. The 'Ignore first line (only used if Variable Names is not empty):' checkbox is unchecked.

IV. DEMO

2. Cấu hình đầu vào

- Vị trí: Randomize Category/Product/Item (JSR223 Sampler)
- Mục đích: Random hợp lệ 3 giá trị categoryId, productId, itemId từ file CSV
- Lý do: Không thể random riêng lẻ vì dữ liệu liên kết với nhau, sẽ sai query trong request

```
1 import java.util.Random
2 import java.nio.file.Files
3 import java.nio.file.Paths
4
5 // 1. Danh sách các category có thể chọn
6 def categories = ['FISH', 'DOGS', 'CATS', 'BIRDS', 'REPTILES']
7
8 // 2. Random 1 category
9 def selectedCategory = categories[new Random().nextInt(categories.size())]
10 vars.put("selectedCategory", selectedCategory)
11
12 // 3. Đọc file CSV & Lọc dòng có category trùng
13 def csvFilePath = "D:/demo-testing/product_summary.csv"
14 def lines = Files.readAllLines(Paths.get(csvFilePath))
15 def headerRemoved = lines.drop(1) // Loại bỏ header nếu có
16 def matched = headerRemoved.findAll { it.split(',')[0] == selectedCategory }
17
18 // 4. Random 1 dòng từ matched
19 if (matched.isEmpty()) {
```

IV. DEMO

3. Các kịch bản kiểm thử

Scenario 1: Browse-Only Load

Bối cảnh:

Test case được thực hiện để đánh giá hiệu năng của hệ thống khi người dùng chỉ duyệt trang web mà không đăng nhập.

Luồng hành vi:

- Truy cập trang chủ (/)
- Truy cập danh mục (ví dụ: */categories/FISH*)
- Chọn 1 sản phẩm (ví dụ: */products/PI-SW-01*)
- Xem chi tiết item (ví dụ: */products/PI-SW-01/items/EST-1*)

Data Set:

- File CSV “product_summary.csv” gồm các cột: *categoryId*, *productId*, *itemId*



IV. DEMO

3. Các kịch bản kiểm thử

Scenario 1: Browse-Only Load

Test case 1: Load Testing

Mục tiêu:

Test case được thực hiện để đánh giá hiệu năng của hệ thống ở **mức tải ổn định**

Thông số:

- Thread Groups - Số lượng users: 40
- Ramp-up time: 30 giây
- Loop: Infiniti
- Duration: 300 giây

Thread Group

Name:

Comments:

- Action to be taken after a Sampler error

☒ Continue ☐ Start Next Thread Loop ☐ Stop Thread

- Thread Properties

Number of Threads (users):

Ramp-up period (seconds):

Loop Count: ☒ Infinite

☒ Same user on each iteration

☐ Delay Thread creation until needed

☒ Specify Thread lifetime

Duration (seconds):

Startup delay (seconds):

IV. DEMO

3. Các kịch bản kiểm thử

Scenario 1: Browse-Only Load

Test case 2: Spike Testing

Mục tiêu:

Test case được thực hiện để đánh giá hiệu năng của hệ thống **khi gặp tải tăng đột ngột**

Dùng Ultimate Thread Group

Kiểm tra chi tiết từng giai đoạn tải: *Bao nhiêu người dùng khởi động? Bao nhiêu người dùng chạy đồng thời?...*

Name:

Scenario 1 - TC02

Comments:

- Action to be taken after a Sampler error

☒ Continue

☐ Start Next Thread Loop

☐ Stop Thread

☐ Stop Test

☐ Stop Test Now

- Threads Schedule

Start Threads Count	Initial Delay, sec	Startup Time, sec	Hold Load For, sec	Shutdown Time
10	0	5	60	10
25	65	10	60	0
70	135	0	20	5
100	220	0	20	5
25	240	5	60	5

IV. DEMO

3. Các kịch bản kiểm thử

Scenario 2: Full Purchase Flow Under Load - Load Testing

Bối cảnh

Test case được thực hiện để đánh giá hiệu năng của hệ thống khi người dùng thực hiện hoàn chỉnh luồng hành vi trong điều kiện tải ổn định

Data Set

- File CSV “users_60.csv” gồm các cột: *username*, *password*
- File CSV “product_summary.csv” gồm các cột: *categoryId*, *productId*, *itemId*

Luồng hành vi

- Truy cập trang chủ (/)
- Truy cập trang login (/account/signonForm)
- Đăng nhập (/account/signon)
- Truy cập danh mục (/categories/{categoryId})
- Truy cập sản phẩm (/products/{productId})
- Truy cập item (/products/{productId}/items/{itemId})
- Thêm vào giỏ (/cart/addItemToCart?itemId=...)
- Truy cập giỏ hàng (/cart/viewCart)
- Mở form đặt hàng (/order/newOrderForm)
- Submit đơn hàng (/order/newOrder)
- Xác nhận đơn hàng (/order/submitOrder)

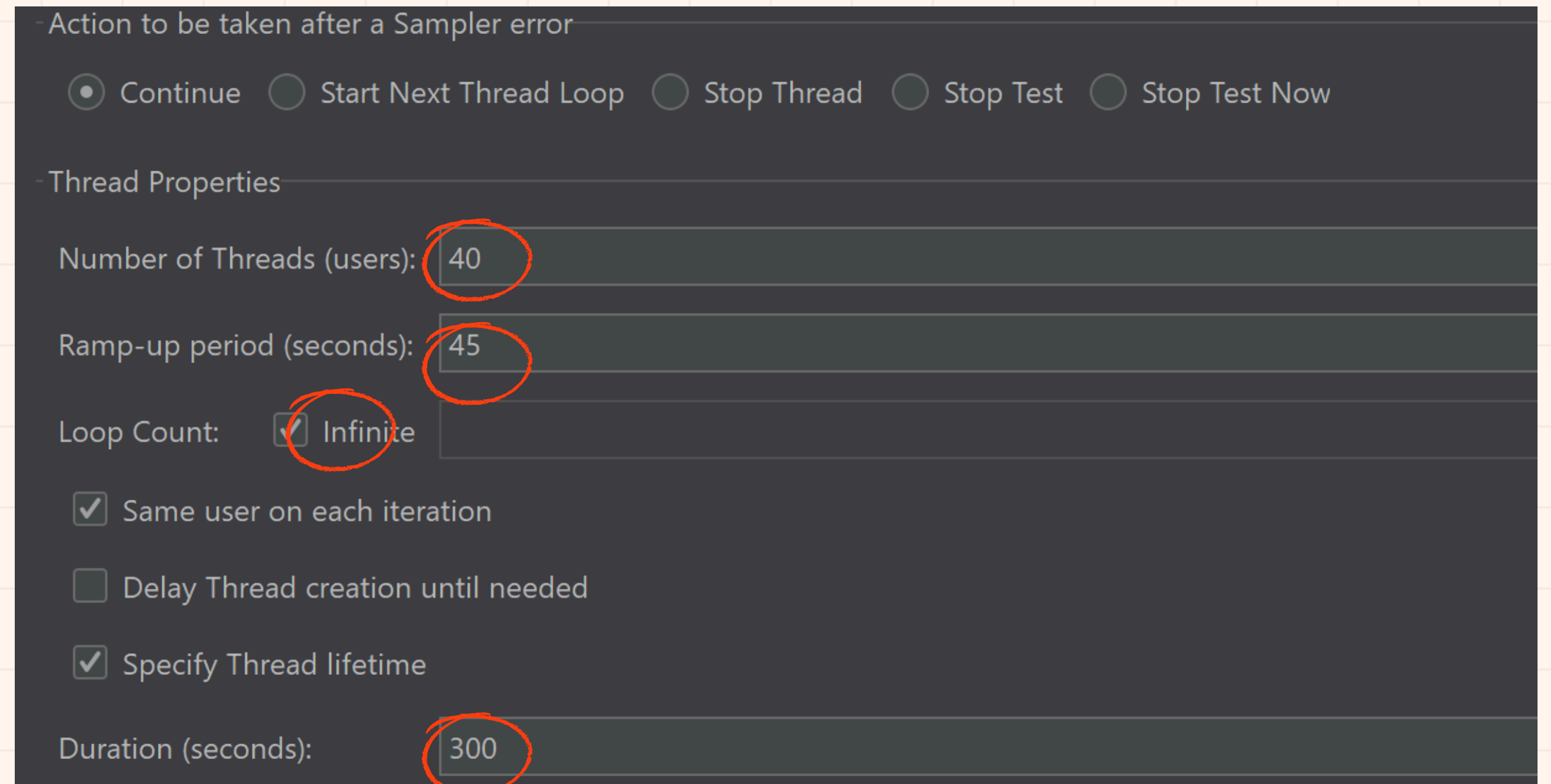
IV. DEMO

3. Các kịch bản kiểm thử

Scenario 2: Full Purchase Flow Under Load - Load Testing

Thông số

- Thread Groups: 40
- Ramp-up time: 45 giây
- Loop: Infinite
- Duration: 300 giây

A screenshot of the JMeter configuration interface for a Thread Group. The 'Action to be taken after a Sampler error' section has 'Continue' selected. The 'Thread Properties' section shows 'Number of Threads (users):' set to 40, 'Ramp-up period (seconds):' set to 45, and 'Loop Count:' set to 'Infinite' (checked). Below these are three checkboxes: 'Same user on each iteration' (checked), 'Delay Thread creation until needed' (unchecked), and 'Specify Thread lifetime' (checked). At the bottom, 'Duration (seconds):' is set to 300. Red circles highlight the values 40, 45, Infinite, and 300.

Action to be taken after a Sampler error

☒ Continue ☐ Start Next Thread Loop ☐ Stop Thread ☐ Stop Test ☐ Stop Test Now

Thread Properties

Number of Threads (users): 40

Ramp-up period (seconds): 45

Loop Count: ☒ Infinite

☒ Same user on each iteration

☐ Delay Thread creation until needed

☒ Specify Thread lifetime

Duration (seconds): 300

IV. DEMO

3. Các kịch bản kiểm thử

Scenario 3: Concurrent Login - Endurance Testing

Bối cảnh:

Test case được thực hiện để đánh giá hiệu năng của hệ thống khi nhiều người dùng liên tục đăng nhập và truy cập tài khoản trong thời gian dài

Luồng hành vi:

- Truy cập trang chủ (/)
- Truy cập trang login (/account/signonForm)
- Đăng nhập (/account/signon)
- Truy cập “My Account” (/account/editAccountForm)
- Đăng xuất (/account/signoff)

Data Set:

- File CSV “users_60.csv” gồm các cột: *username*, *password*



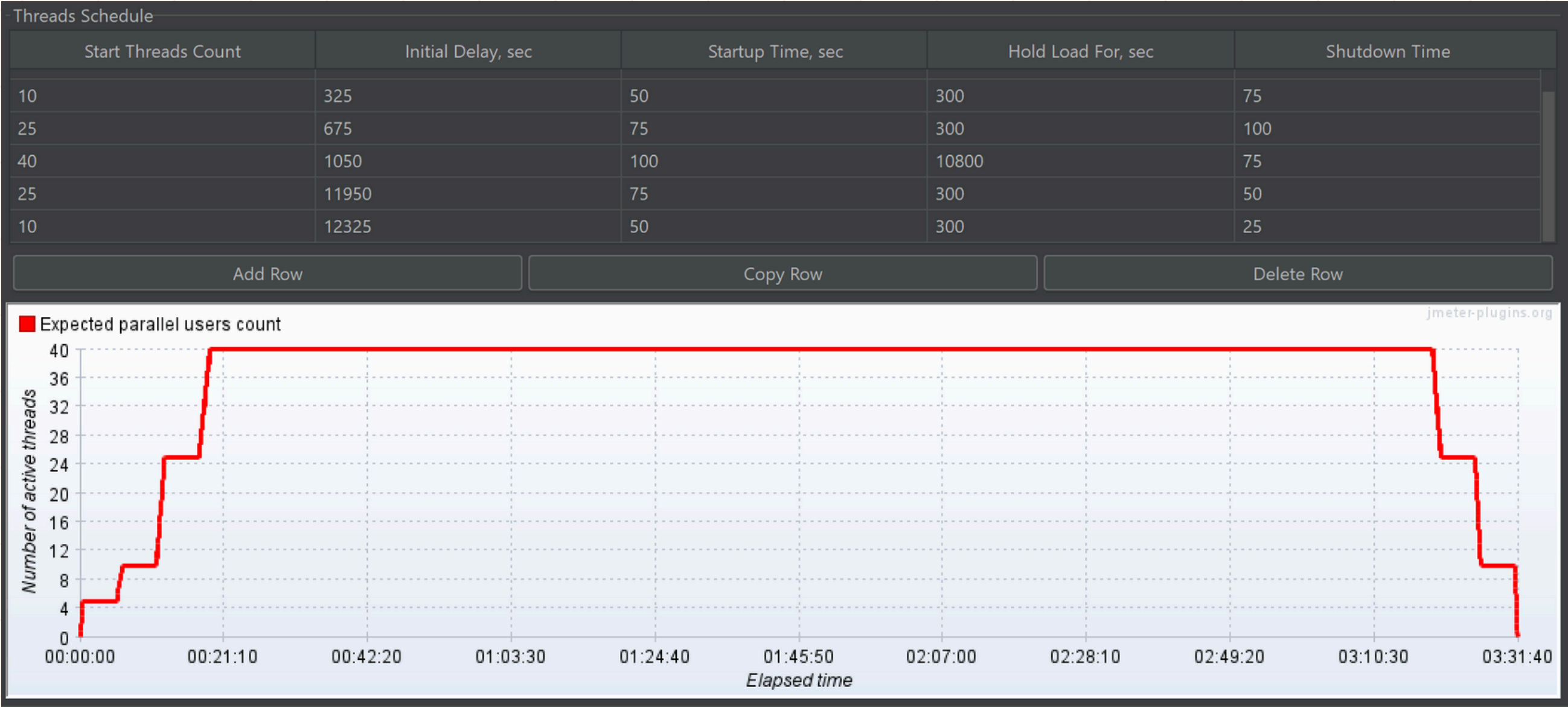
IV. DEMO

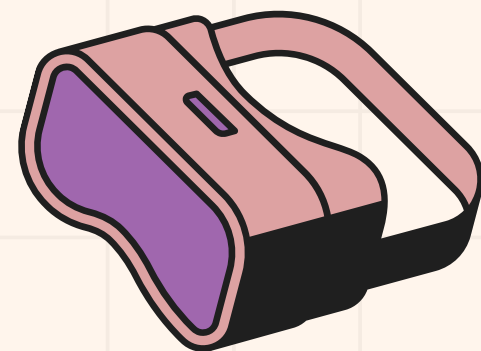
3. Các kịch bản kiểm thử

Scenario 3: Concurrent Login - Endurance Testing

Dùng Ultimate Thread Group

Kiểm tra hệ thống có bị memory leak, crash hay lỗi session sau thời gian dài hoạt động.
Tổng số thread tối đa là 40, chạy liên tục hơn 3 tiếng.





THANK YOU

